Lecture 11. Complexity of Counting

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Function Problems

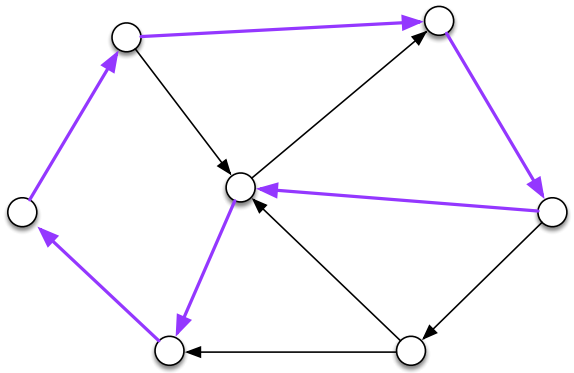
- So far all complexity classes are defined on decision problems:
  - P, NP, PSPACE, BPP, etc
- Now we consider function problems:
  - Given x, compute f(x) (*usually output is an integer*)
- Class FP: set of functions computable by a deterministic polynomial time TM
  - Analog of P for function problems



Counting problems -- Examples

- #SAT: given a 3-CNF φ, how many satisfying assignments are there?

- #CYCLE: given a directed graph G, the number of simple cycles in G





#P

- #P is the class of function problems expressible as:

  given x, $f(x) = |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$

  - p(·) is a polynomial
  - M(x,y) is a polynomial-time TM

How many certificates are there

- compare to NP:

  $x \in L \Leftrightarrow \exists y \in \{0,1\}^{p(|x|)}$ such that $M(x, y)=1$

Whether there exists a certificate



#P

- #P is the class of function problems expressible as:

  given x, $f(x) = |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$

  - p(·) is a polynomial
  - M(x,y) is a polynomial-time TM

  - #SAT, #CYCLE are in #P

How many certificates are there



- Of course #SAT is NP-hard

- Are these problems hard?
  - *(decision problems are in P)*

    #SHORTEST-PATH

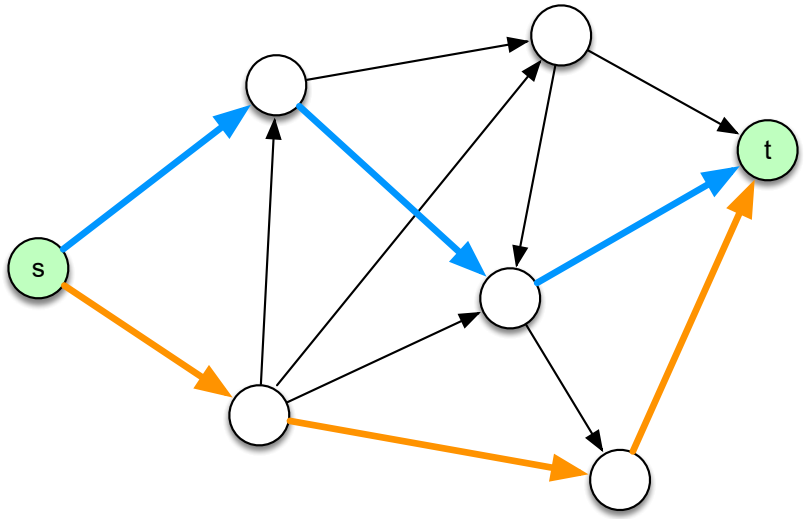
    #MATCHING

    #CYCLE



#SHORTEST-PATH

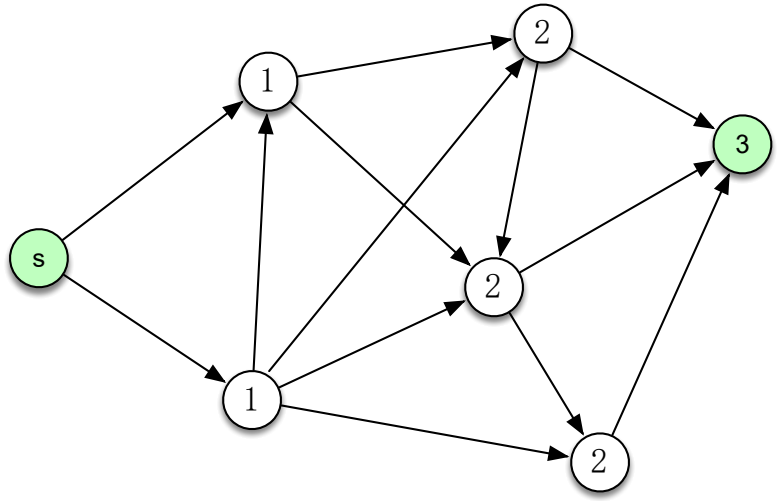
- In an unweighted directed graph G, how many shortest paths between s and t?





#SHORTEST-PATH

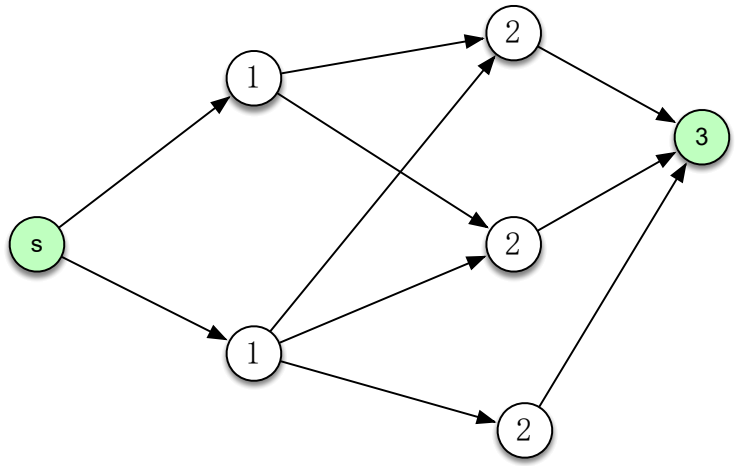
- First we partition the vertices by the distance from s:





#SHORTEST-PATH

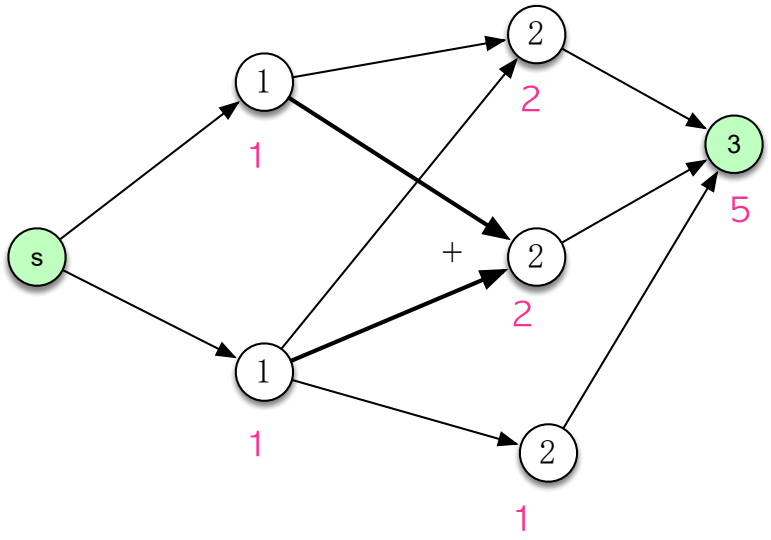
- First we partition the vertices by the distance from s
- Only keep edges on some shortest paths from s:





#SHORTEST-PATH

- First we partition the vertices by the distance from s
- Only keep edges on some shortest paths from s
- Then a dynamic programming by the order of distances





- Of course #SAT is NP-hard

- Are these problems hard?
  - *(decision problems are in P)*

    #SHORTEST-PATH

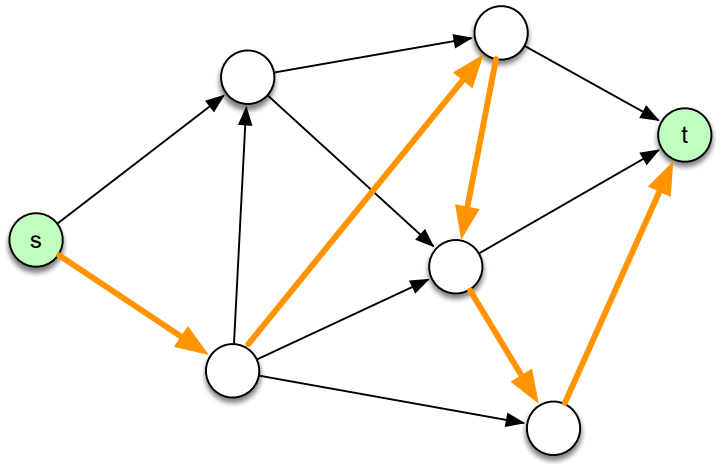


What about #PATH and #CYCLE ?



#PATH

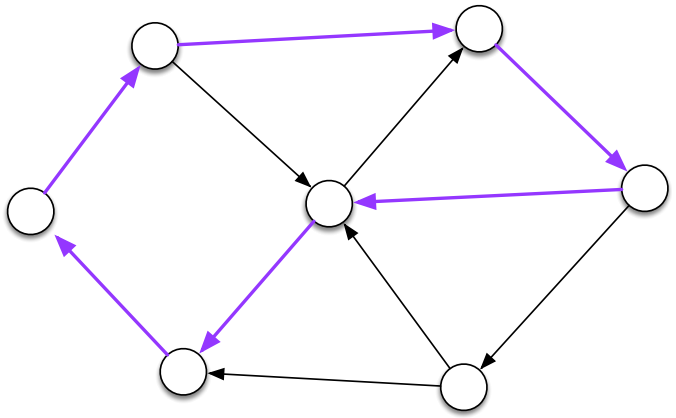
- In a directed graph G, how many simple paths between s and t?





#CYCLE

- In a directed graph G, how many simple cycles in G?
  - Finding a cycle is in linear time by DFS
  - However, counting is not easy

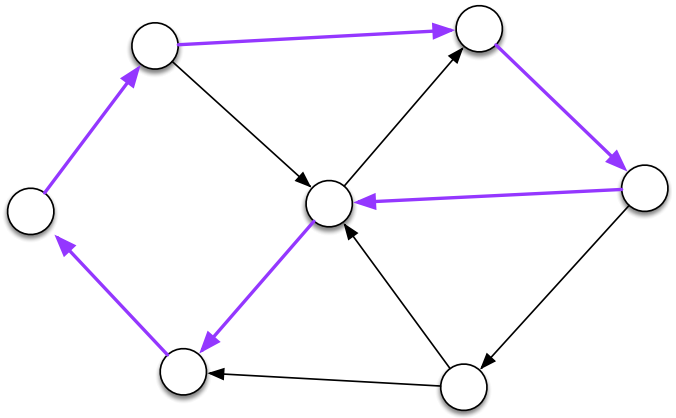




#CYCLE

- In a directed graph G, how many simple cycles in G?

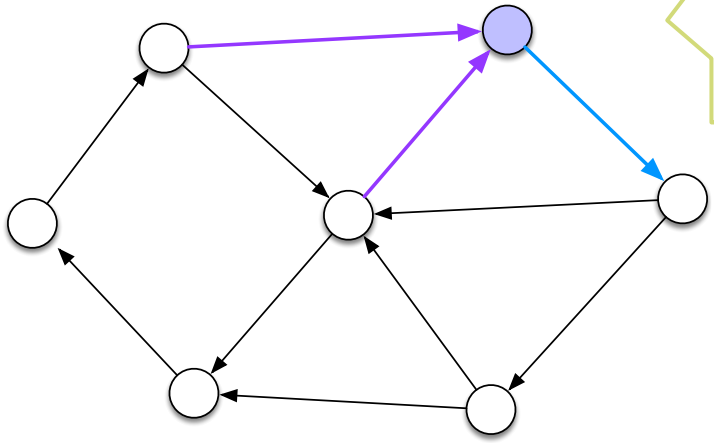
- we can reduce Hamiltonian cycle to #CYCLE
  - So if #CYCLE is in FP, P=NP

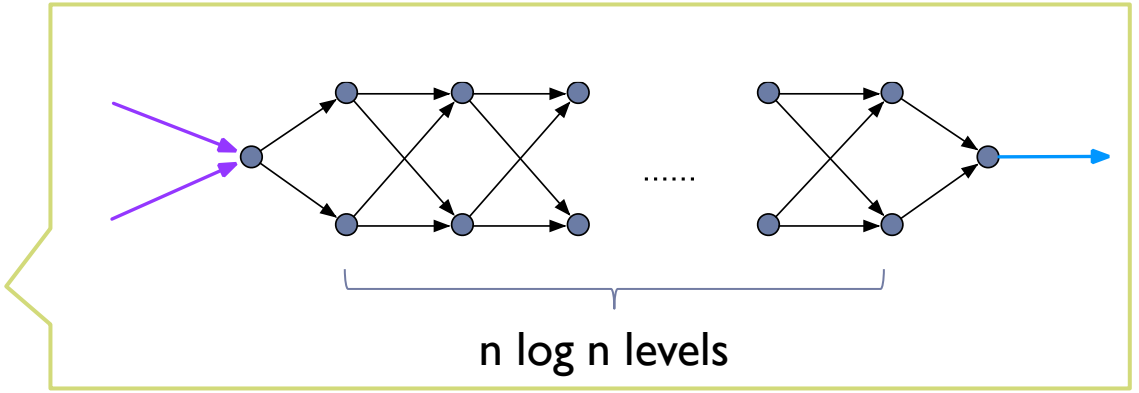




Hamiltonian cycle ⟹ #CYCLE

- Replace every vertex by the gadget, obtain G':



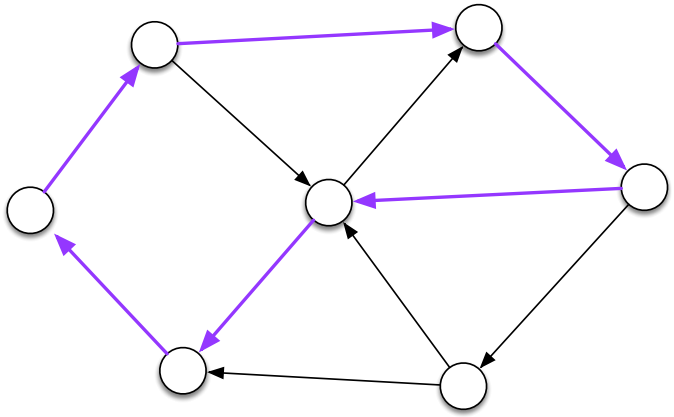


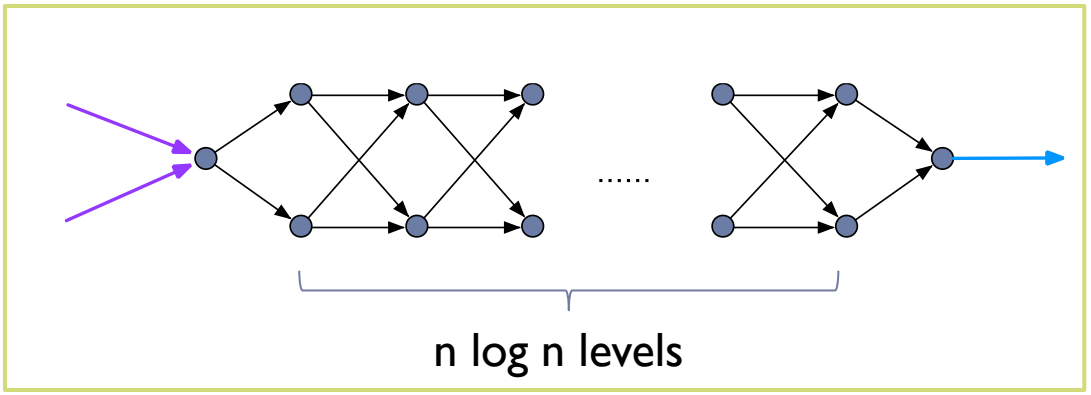
#PATH in the gadget:

$2^{n \log n} = n^n$



Hamiltonian cycle ⟹ #CYCLE



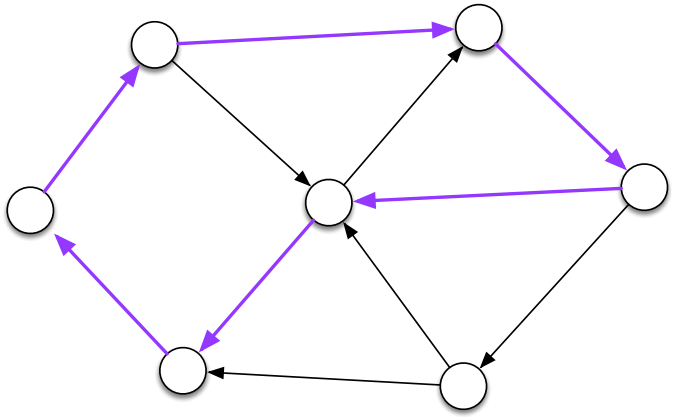


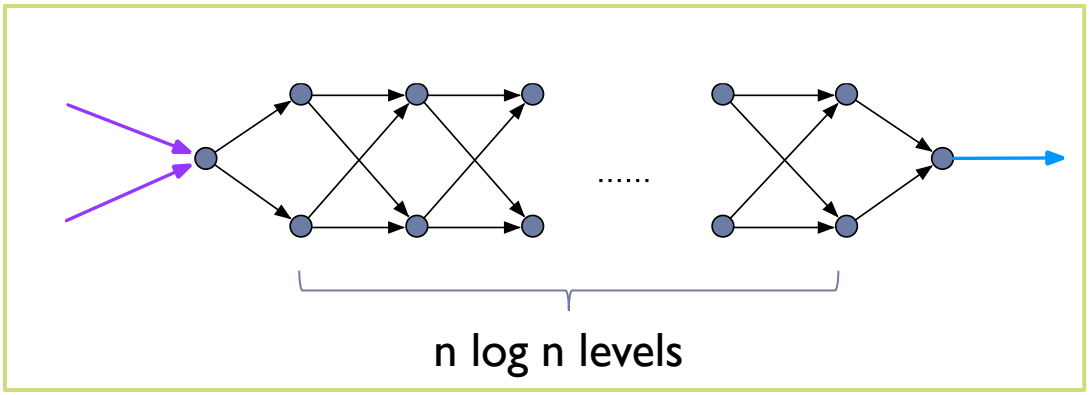
#PATH in the gadget:
$2^{n \log n} = n^n$

- So a path of length k in G will yield $(n^n)^k$ paths in G'



Hamiltonian cycle ⟹ #CYCLE





#PATH in the gadget:
$2^{n \log n} = n^n$

- So a path of length k in G will yield $(n^n)^k$ paths in G'
  - G has Hamiltonian cycle ⇒ G' has $\geq n^{n^2}$ cycles
  - G has no Hamiltonian cycle ⇒ G has $\leq n^{n-1}$ cycles ⇒ G' has $\leq n^{n-1} \cdot (n^n)^{n-1} < n^{n^2}$ cycles



#CYCLE

- In a directed graph G, how many simple cycles in G?

- we can reduce Hamiltonian cycle to #CYCLE
  - So if #CYCLE is in FP, P=NP
  - #PATH is similarly NP-hard



- Of course #SAT is NP-hard

- Are these problems hard?
  - *(decision problems are in P)*

#SHORTEST-PATH

in FP

#MATCHING

#PATH

#CYCLE

NP-hard



PP: *decision problem analog for #P*

- #P is the class of function problems expressible as:

  given x, $f(x) = |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$

  - p(·) is a polynomial
  - M(x,y) is a polynomial-time TM

- PP: $x \in L \Leftrightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}| \geq \frac{1}{2} \cdot 2^{p(|x|)}$
  - "a majority of strings are certificates"

Is PP easier than #P?



PP=P $\Leftrightarrow$ #P=FP

- If PP=P, how to design a polynomial time algorithm for #P?

#P is the class of function problems expressible as:
given x, $f(x) = |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$
where M is a polynomial time TM



PP=P ⟺ #P=FP

- If PP=P, how to design a polynomial time algorithm for #P?

> #P is the class of function problems expressible as:
> given x, $f(x) = |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$
> where M is a polynomial time TM

  - We make another TM N_u for some integer u:
  - $N_u(x,y)$ accepts iff y<u (y∈$\{0,1\}^{p(|x|)}$ and treated as binary number)
  - So $\#N_u(x,y)=u$
  - Define $M_u(x, \overline{y0}) = M(x, y), M_u(x, \overline{y1}) = N_u(x, y)$
  - Then $\#M_u=\#M+\#N_u$
  - By PP oracle we can check whether $\#M+u \geq 2^{p(|x|)}$
  - So by binary search we can find #M (in polynomial steps)



PP and BPP?

- **BPP** (Bounded-error Probabilistic Poly-time)
  - L ∈ **BPP** if there is a p.p.t. TM M:

$$x \in L \Rightarrow \Pr_y[M(x,y)=1] \geq 2/3$$

$$x \notin L \Rightarrow \Pr_y[M(x,y)=1] \leq 1/3$$

- Equiv. to:

$$x \in L \Rightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x,y)=1\}| \geq \frac{2}{3} \cdot 2^{p(|x|)}$$

$$x \notin L \Rightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x,y)=1\}| \leq \frac{1}{3} \cdot 2^{p(|x|)}$$

- PP: $x \in L \Leftrightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x,y)=1\}| \geq \frac{1}{2} \cdot 2^{p(|x|)}$
- So BPP has a gap, but PP doesn't



#P-complete

- problem f is **#P**-complete if
  - f is in **#P**
  - every problem in **#P** reduces to f

- How to define these kind of reduction?
  - We can use oracle TM
  - (generalized to function problems)



#P-complete

- problem f is **#P**-complete if
  - f is in **#P**
  - every problem in **#P** reduces to f
  - every problem g in **#P** is in $\mathbf{FP}^f$

- If f is **#P**-complete and f∈**FP**, then **#P=FP**



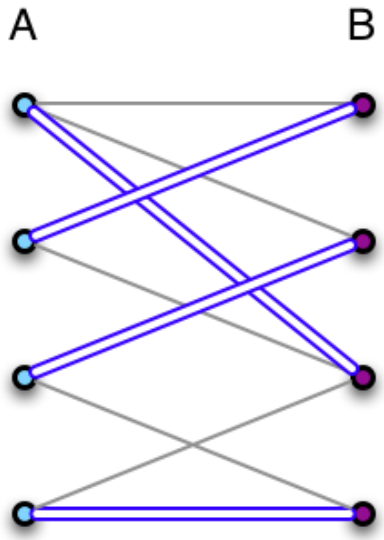
- parsimonious reduction: bijective reduction
  - many standard **NP**-completeness reductions are parsimonious
  - therefore: if #SAT is **#P**-complete we get lots of **#P**-complete problems
- You can check the Levin reduction is parsimonious
- So #SAT is **#P-complete**
  - In fact, counting versions of many NP-complete languages such as 3SAT, Hamiltonian Cycle, and CLIQUE naturally lead to #P-complete problems
  - Counting versions of P problems?



Perfect matching

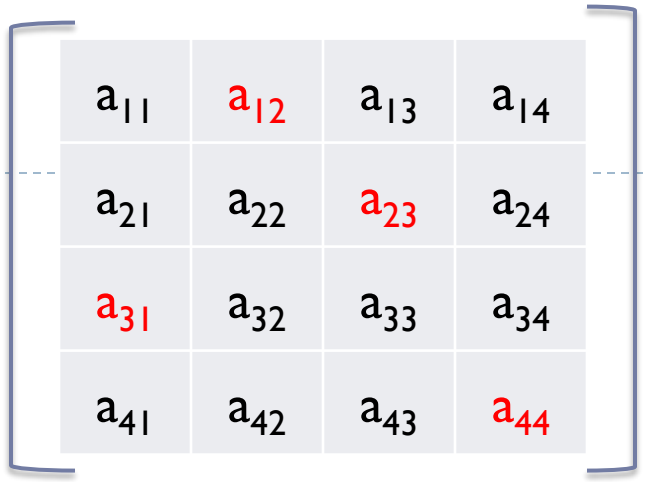
- Perfect matching in bipartite graph:
  - Whether there is a perfect matching in G

- #PM
  - number of perfect matchings in G





Another example



- #PM
  - number of perfect matchings in G

- by determinant of adjacency matrix

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma_i}$$

  - σ is the permutation of {1,…,n}
  - sgn(σ) is +1 or -1



Another example

- #PM
  - number of perfect matchings in G

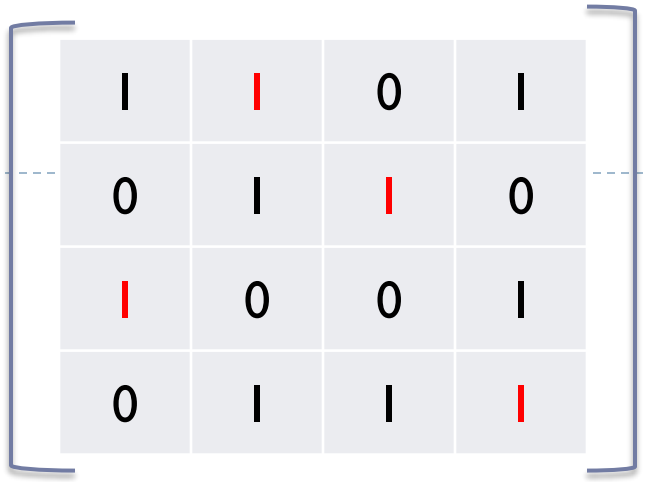
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}

- Permanent of A:

$$\mathrm{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma_i}$$



Another example



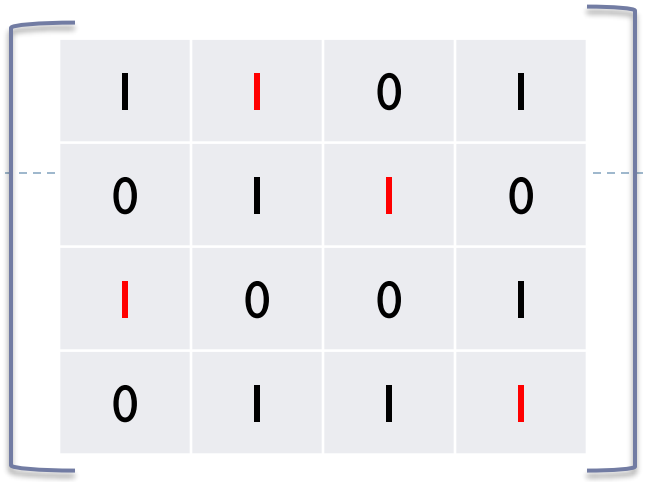
- #PM
  - number of perfect matchings in G
- Permanent of A:

$$\mathrm{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma_i}$$

  - Then perm(A) is just the number of perfect matchings in G



Another example



- #PM
  - number of perfect matchings in G

- Permanent of A:

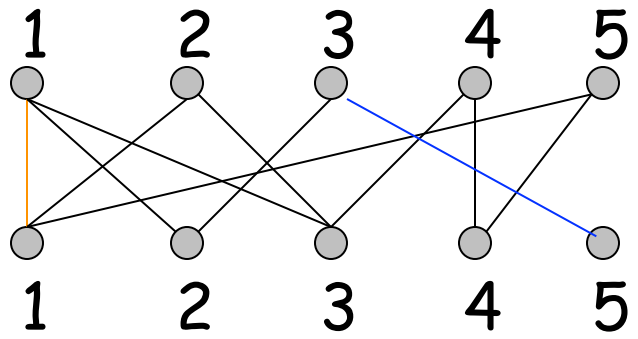
$$\mathrm{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma_i}$$

- determinant is in P, but permanent is #P-complete
- perfect matching is in P, but #PM is #P-complete

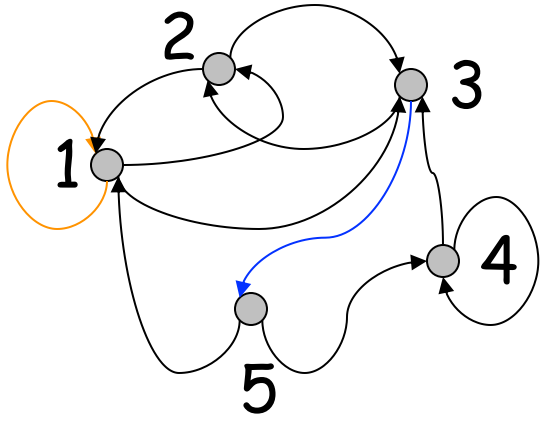


Cycle cover

- Bipartite graph G ⇔ Directed graph G'



G = (U, V, E)

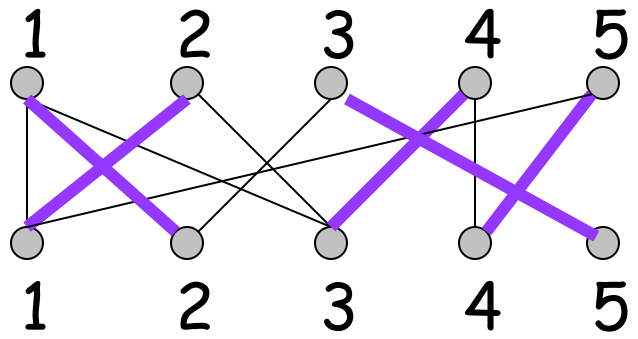


G' = (V, E')

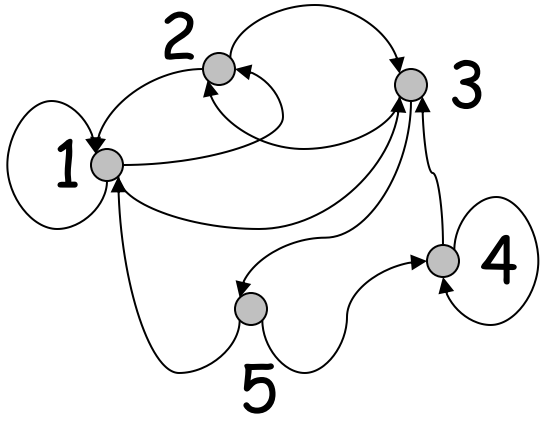


Cycle cover

- Bipartite graph G ⟺ Directed graph G'
  - Perfect matching in G ⟺ Cycle cover in G'



G = (U, V, E)

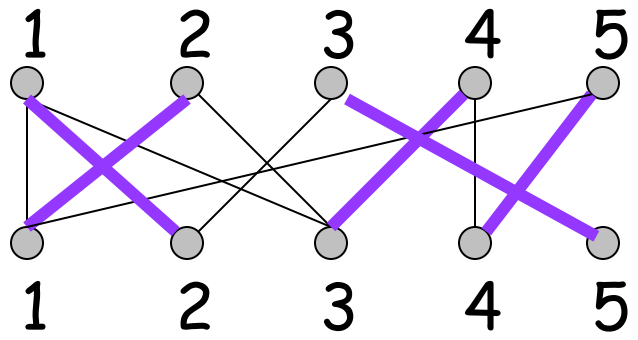


G' = (V, E')

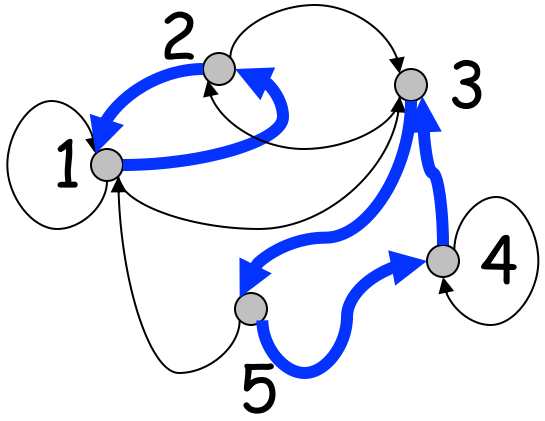


Cycle cover

- Bipartite graph G ⟺ Directed graph G'
  - Perfect matching in G ⟺ Cycle cover in G'
  - cycle cover: collection of node-disjoint directed cycles that touch every node
  - #CYCLE-COVER: given directed graph G = (V, E), how many cycle covers does it have?



G = (U, V, E)



G' = (V, E')



Cycle cover

- Bipartite graph G ⟺ Directed graph G'
  - Perfect matching in G ⟺ Cycle cover in G'
  - cycle cover: collection of node-disjoint directed cycles that touch every node
  - #CYCLE-COVER: given directed graph G = (V, E), how many cycle covers does it have?

  - So Permanent, #PM and #CYCLE-COVER parsimoniously reducible to each other



Permanent ⟺ Weighted Cycle Cover

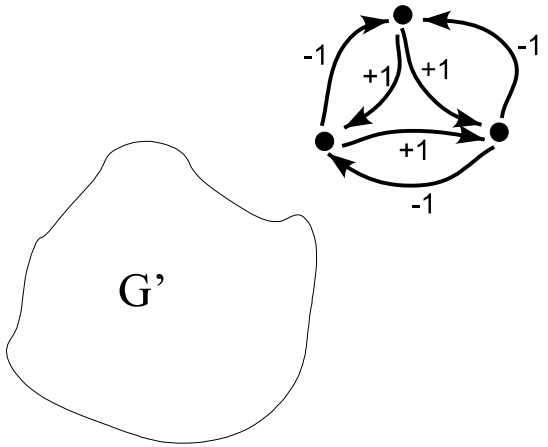
- Introduce edge weights
  - cycle cover weight is product of weights of its edges
  - Weighted cycle cover: sum of weights of all cycle covers
  - Equal to the permanent of its adjacency matrix

$$\mathrm{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^{n} a_{i,\sigma_i}$$



Permanent ⟺ Weighted Cycle Cover

Weighted cycle cover?

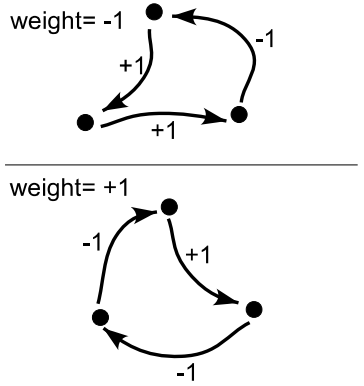


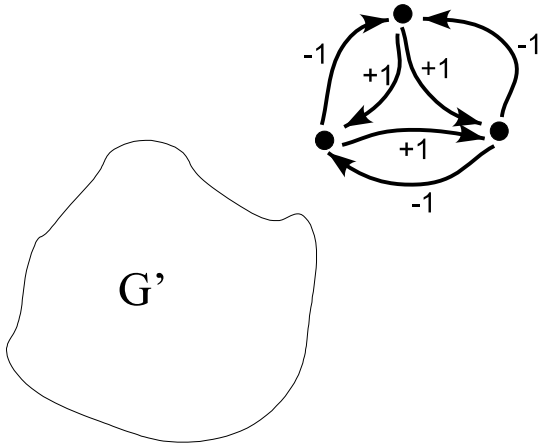


Permanent ⟺ Weighted Cycle Cover

Weighted cycle cover?

0

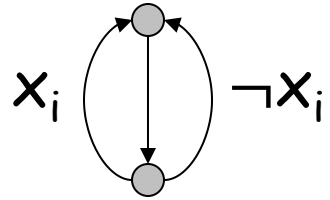






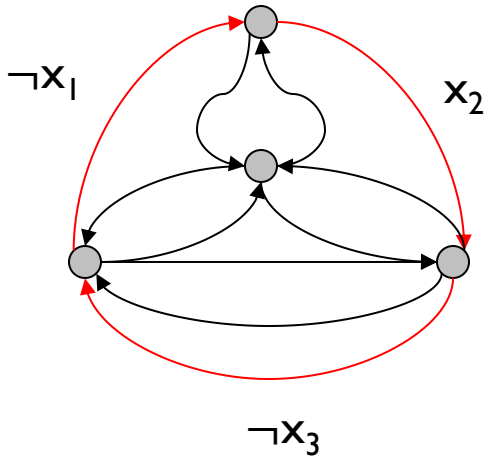
#CYCLE-COVER is #P-complete

- Reduce #3SAT to #(Weighted-)CYCLE-COVER
- variable gadget: every cycle cover includes left cycle or right cycle



- clause gadget:

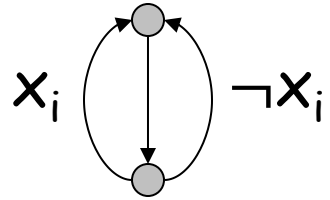
$$(\neg x_1 \vee x_2 \vee \neg x_3)$$





#CYCLE-COVER is #P-complete

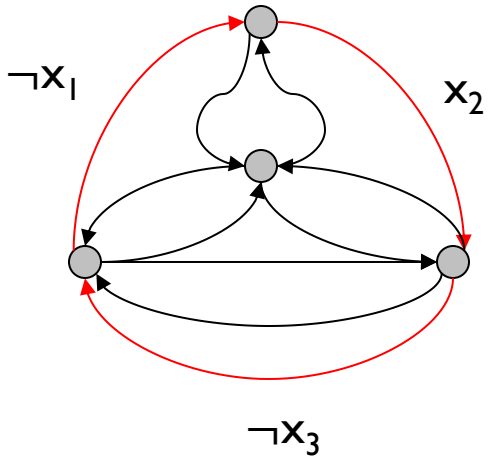
- Reduce #3SAT to #(Weighted-)CYCLE-COVER
- variable gadget: every cycle cover includes left cycle or right cycle



- clause gadget:

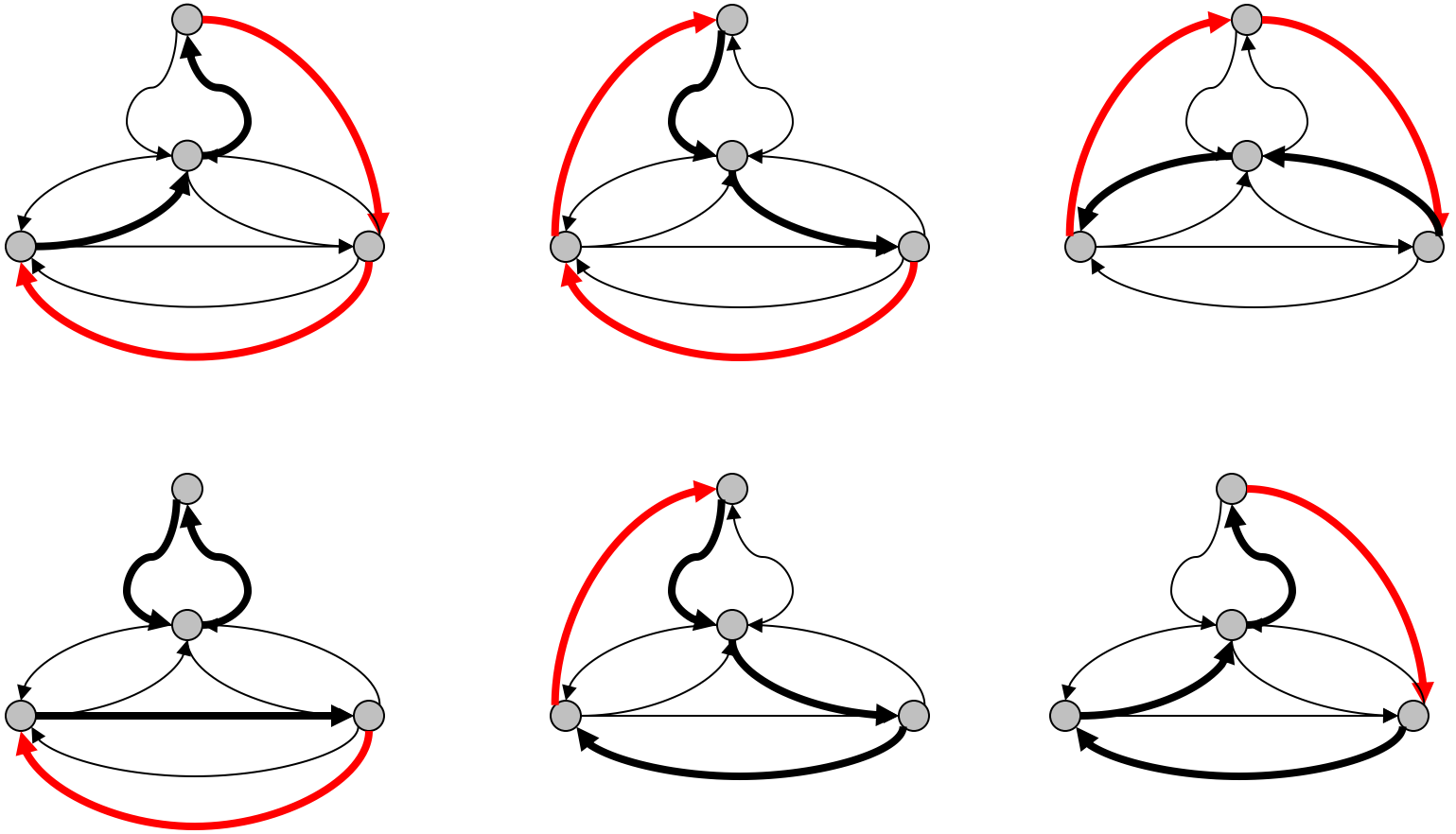
  cycle cover cannot use all three outer edges

  – and each of other 7 combinations gives exactly 1 cover using those external edges



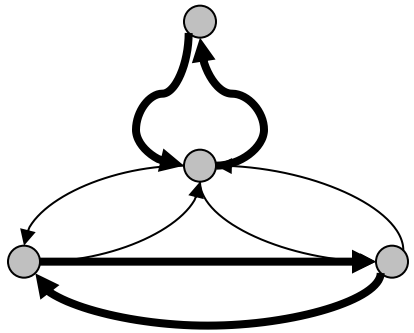


Clause gadget





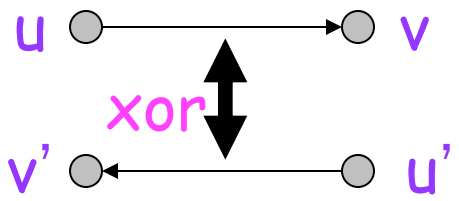
Clause gadget

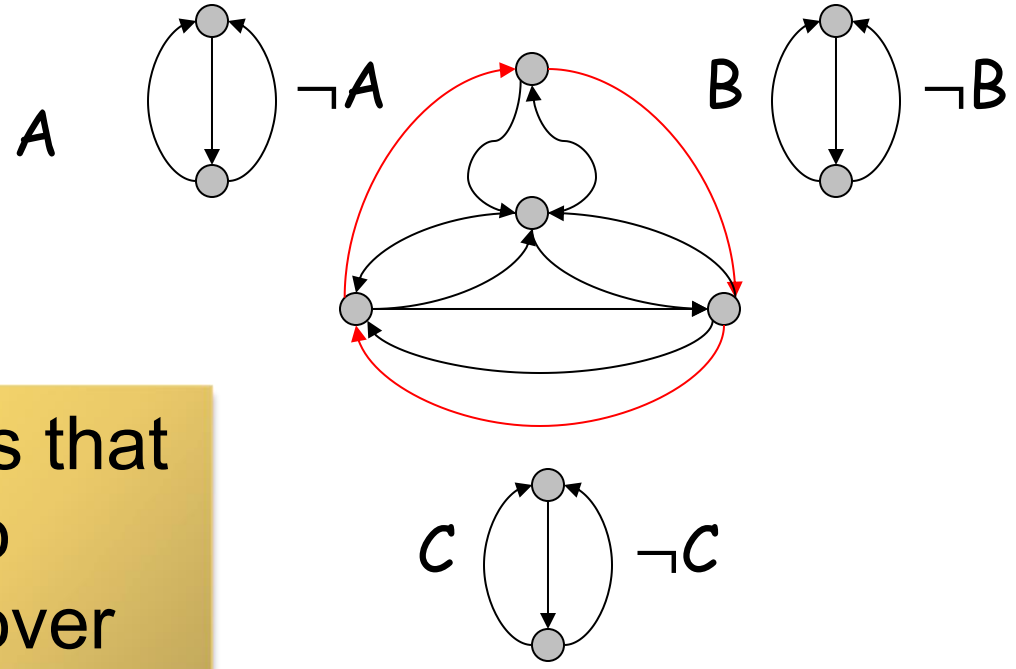




#CYCLE-COVER is #P-complete

- clause gadget corresponding to (AVBVC) has "xor" gadget between outer 3 edges and A, B, C





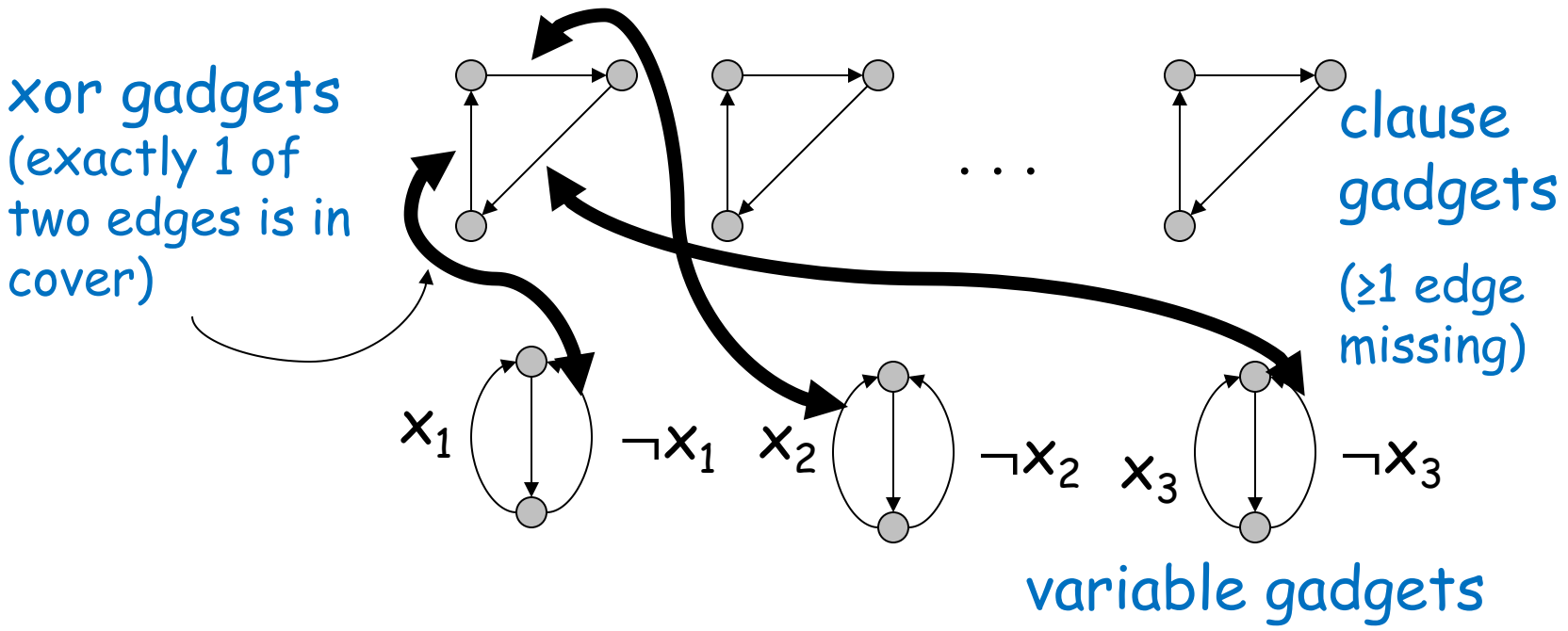
xor gadget ensures that **exactly one** of two edges can be in cover



Reduction

- Proof outline (reduce from #SAT)

  $(\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_1) \wedge \ldots \wedge (x_3 \vee \neg x_2)$





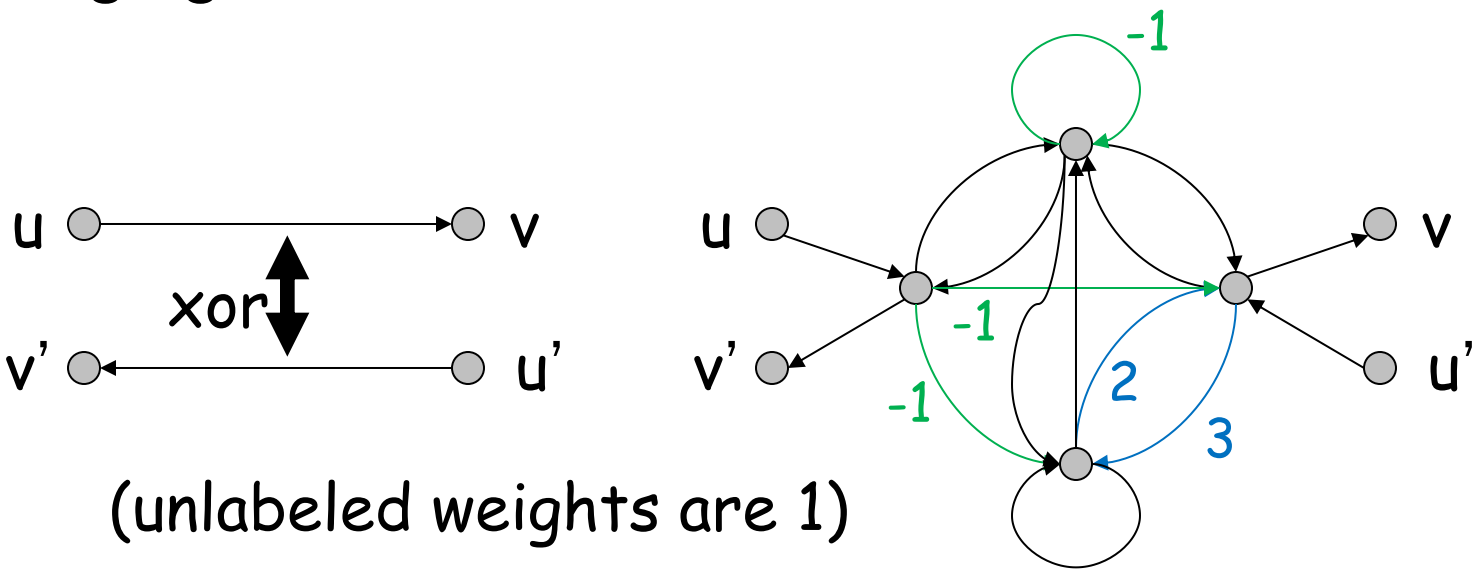
XOR gadget

- "implement" xor gadget by
  - weight of cycle cover that "obeys" xor multiplied by 4
  - weight of cycle cover that "violates" xor multiplied by 0



XOR gadget

- Weighted xor gadget:

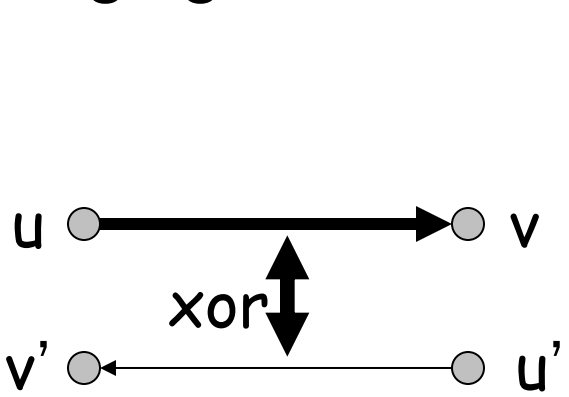


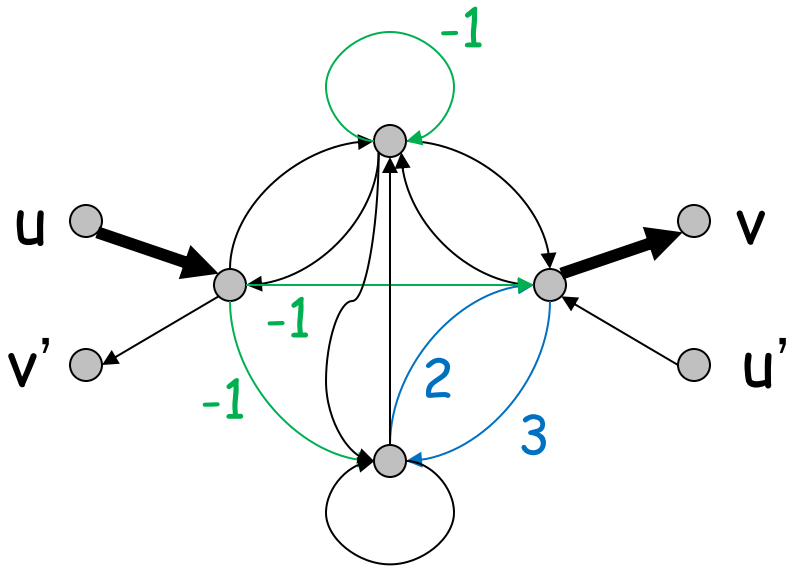
  - weight of cycle cover that "obeys" xor multiplied by 4
  - weight of cycle cover that "violates" xor multiplied by 0



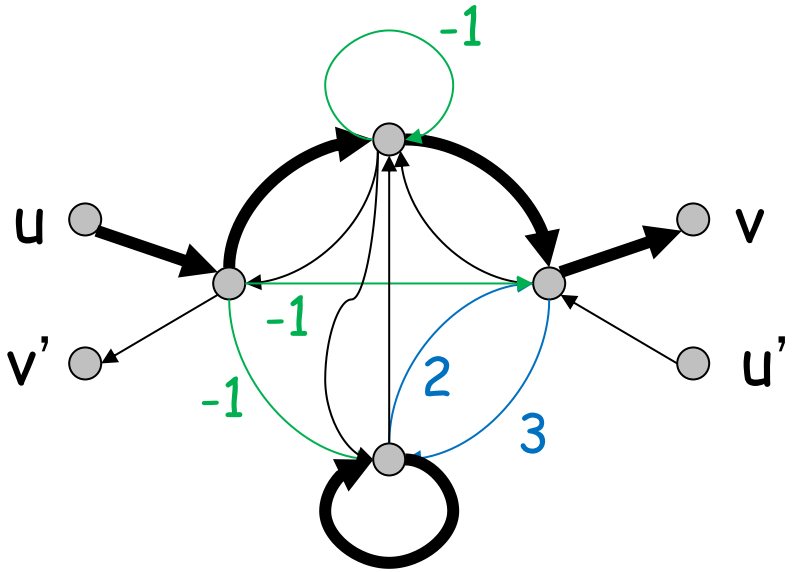
Cycle Cover is #P-complete

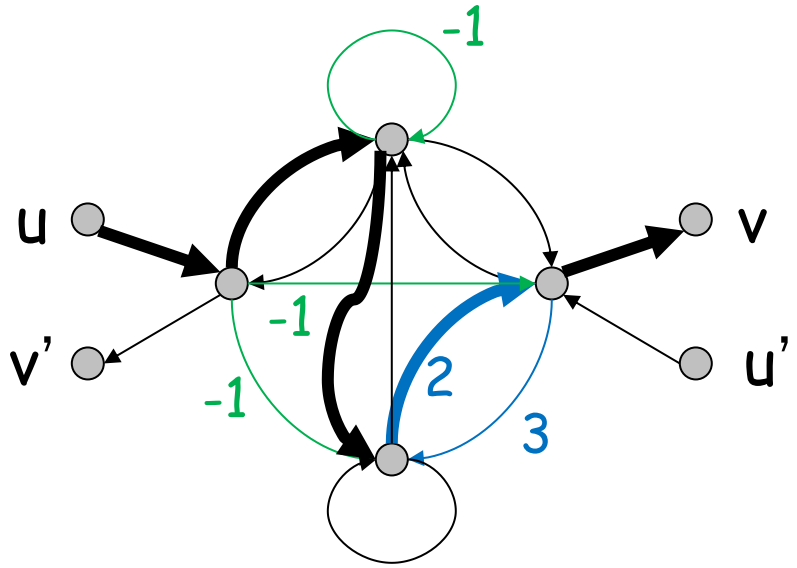
- Weighted xor gadget:

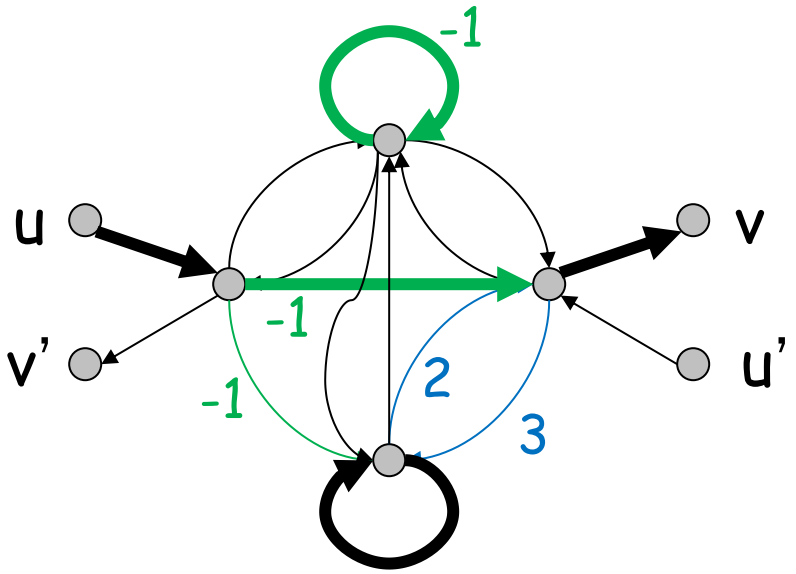


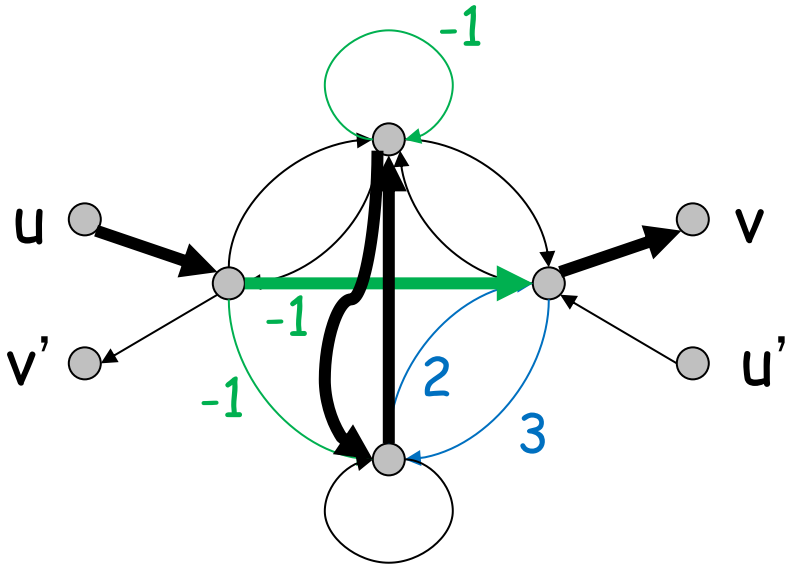




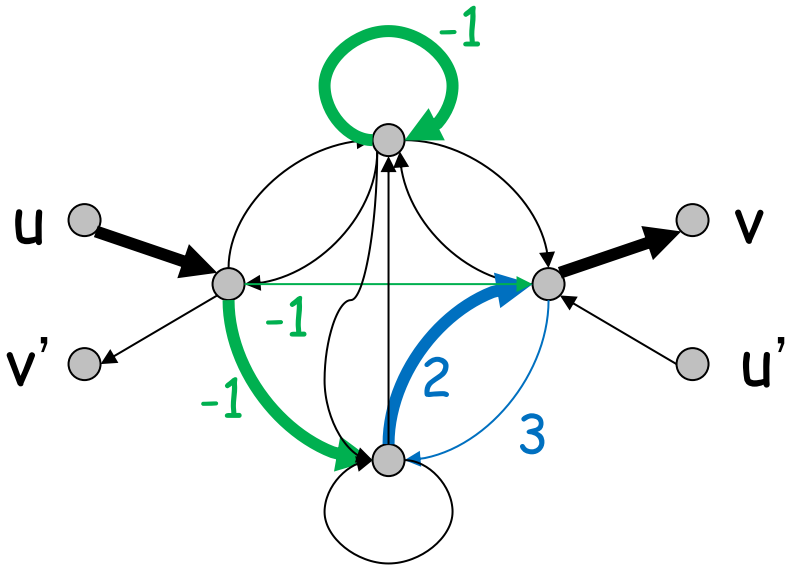

-1
u
v
v'
u'
-1
-1
2
3

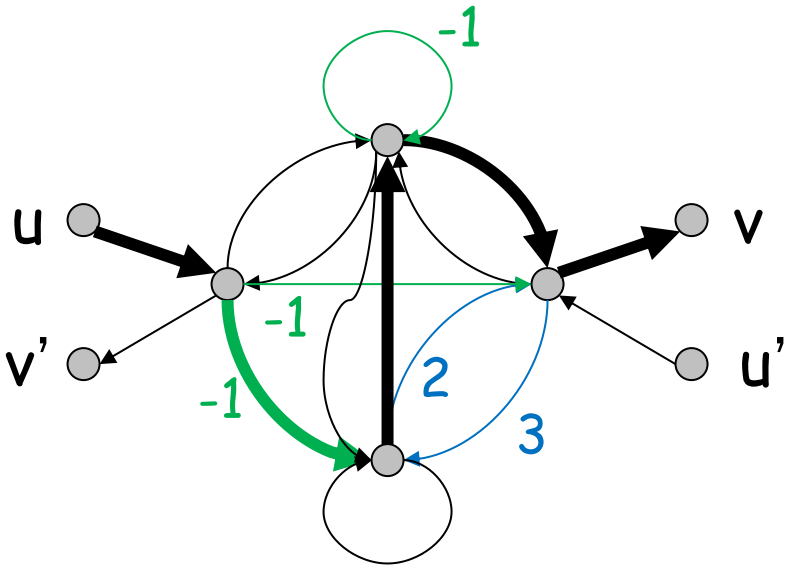

-1
u
v
v'
u'
-1
-1
2
3


-1
u
v
v'
u'
-1
-1
2
3


-1
u
v
v'
u'
-1
-1
2
3



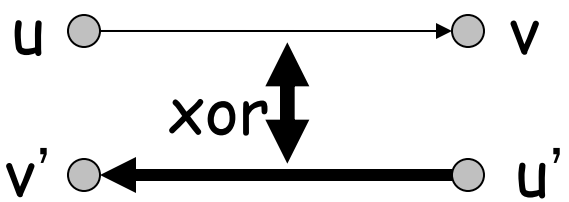

-1
u
v
v'
u'
-1
-1
2
3

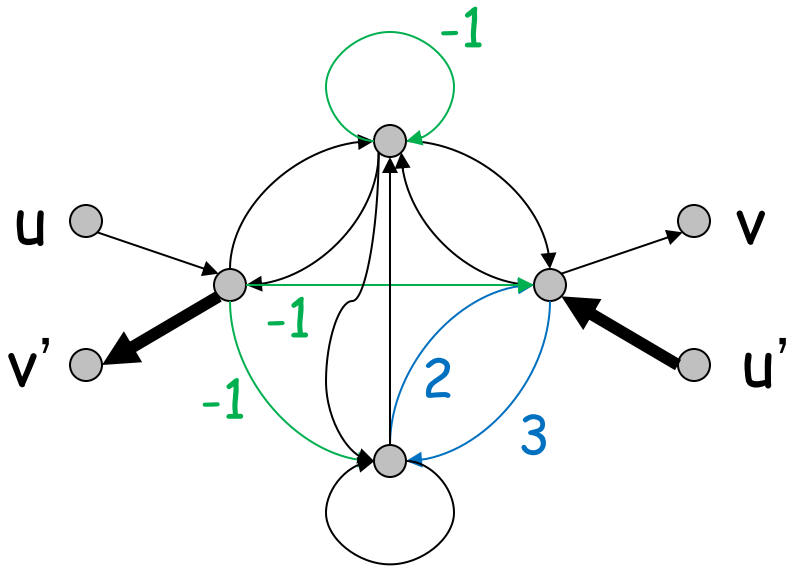

-1
u
v
v'
u'
-1
-1
2
3



XOR gadget

- Weighted xor gadget:

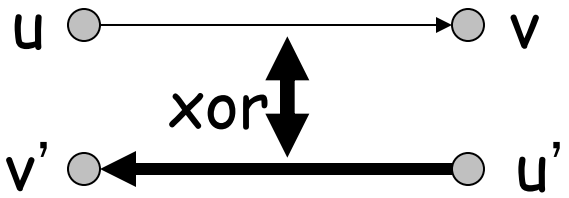


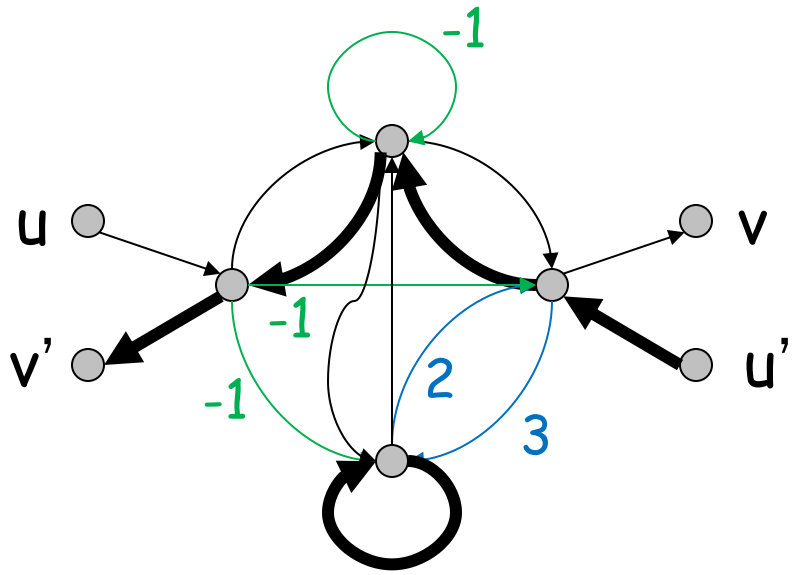


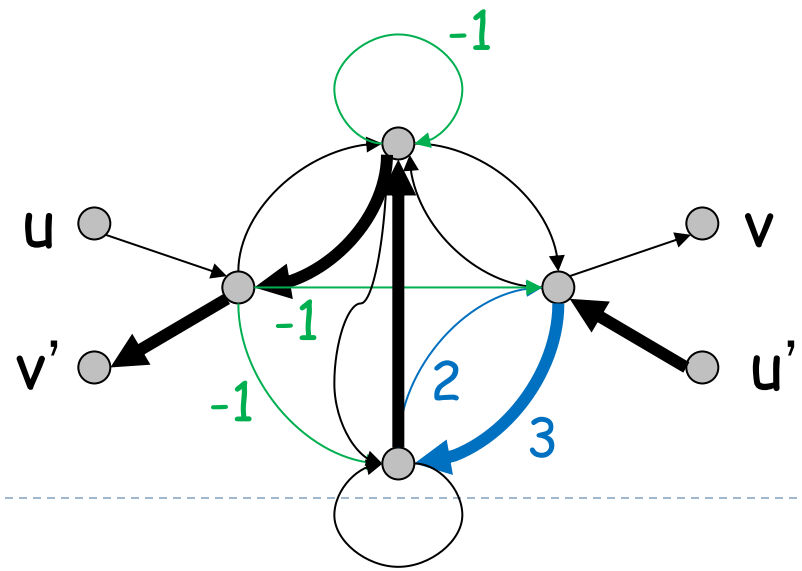


XOR gadget

- Weighted xor gadget:



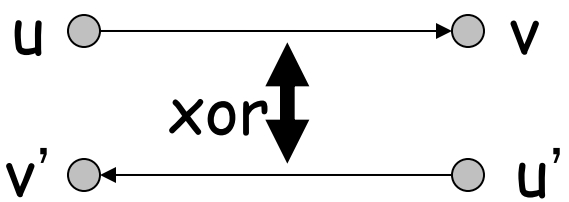


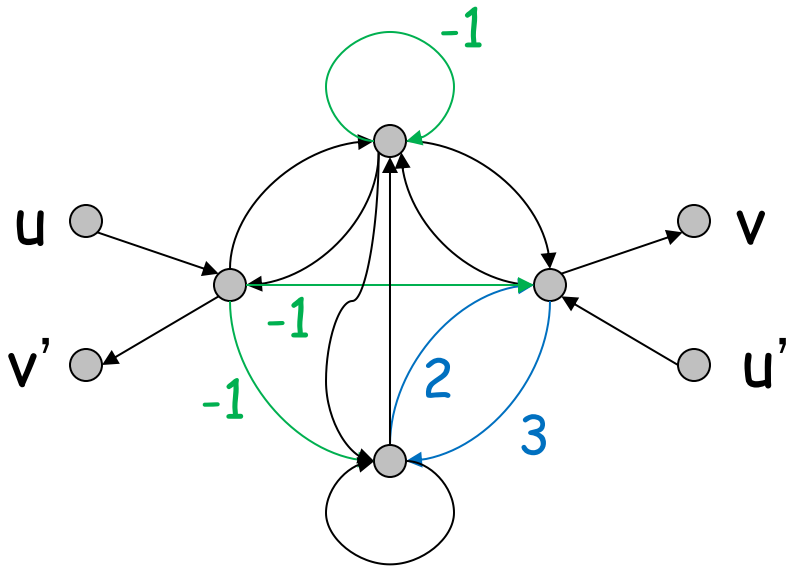




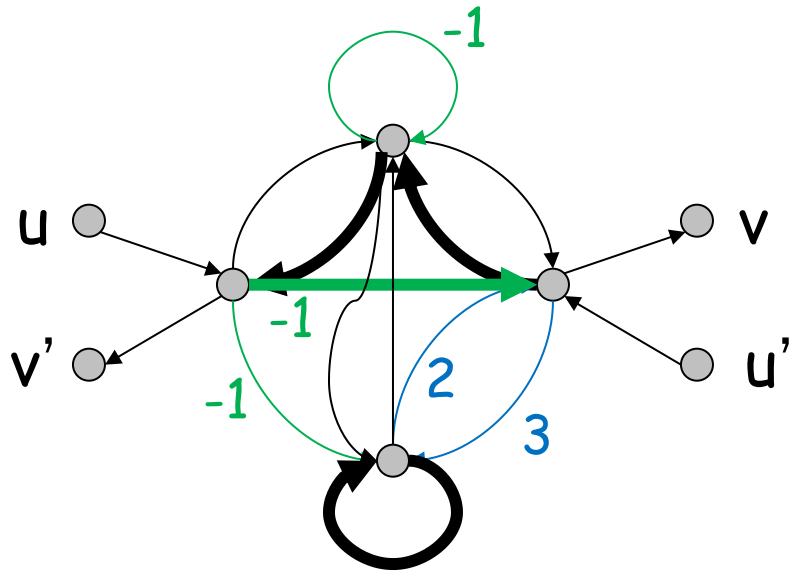
XOR gadget

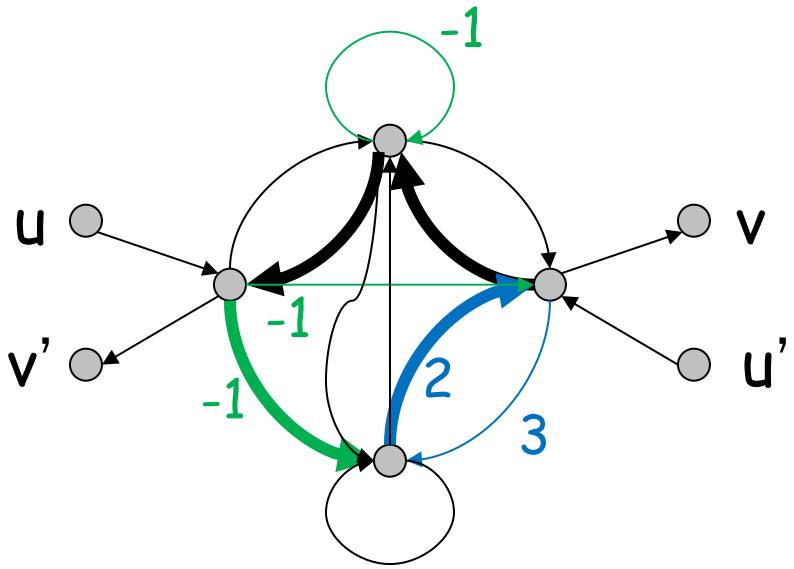
- Weighted xor gadget:

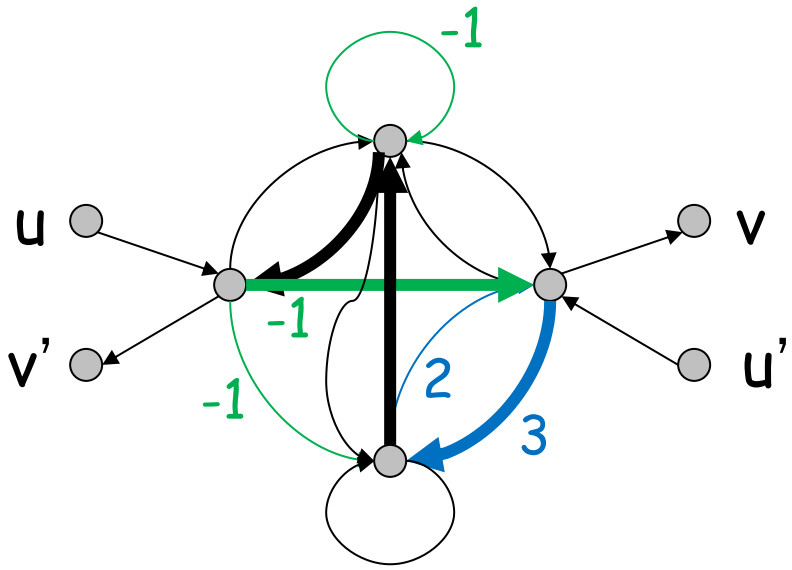


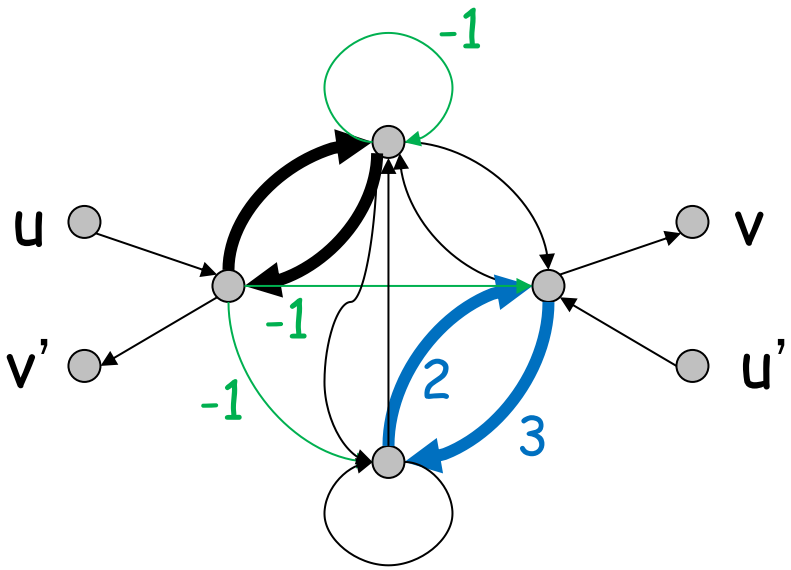





-1
u
v
v'
u'
-1
-1
2
3


-1
u
v
v'
u'
-1
-1
2
3

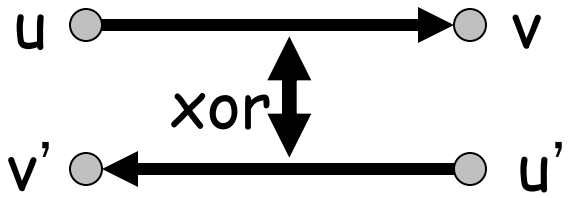

-1
u
v
v'
u'
-1
-1
2
3

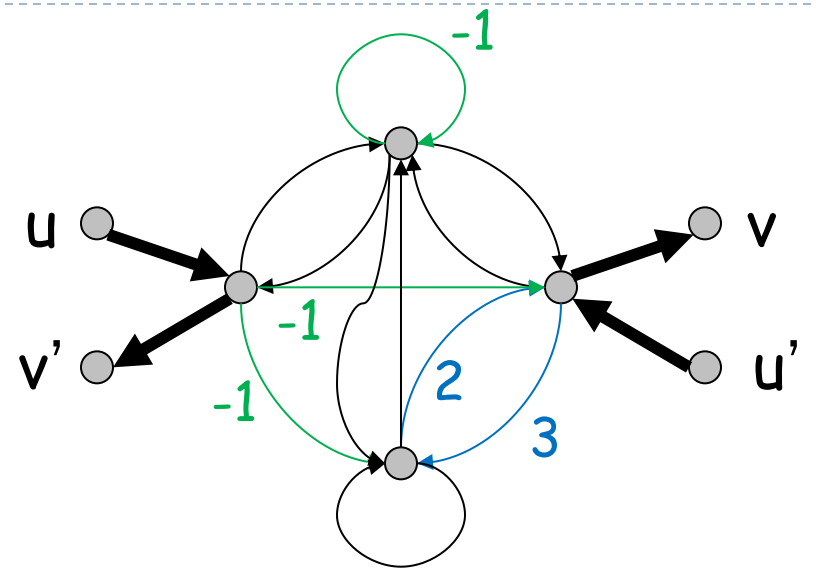

-1
u
v
v'
u'
-1
-1
2
3

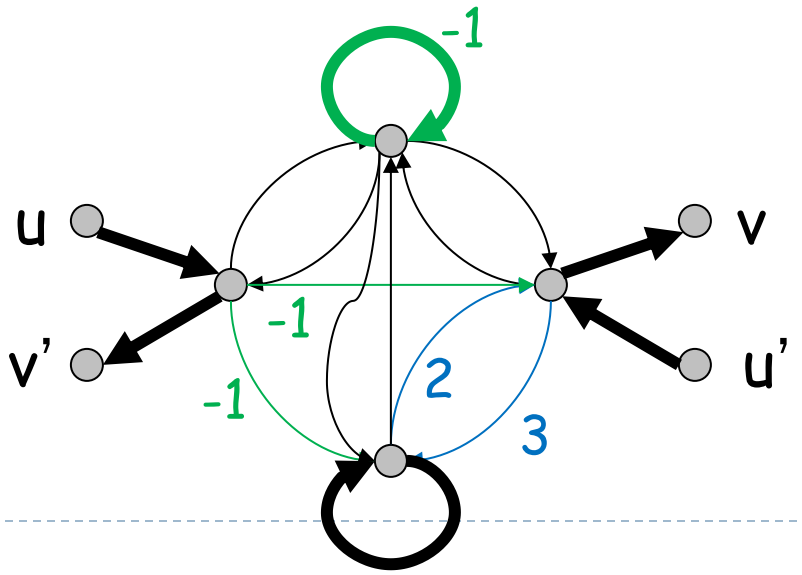


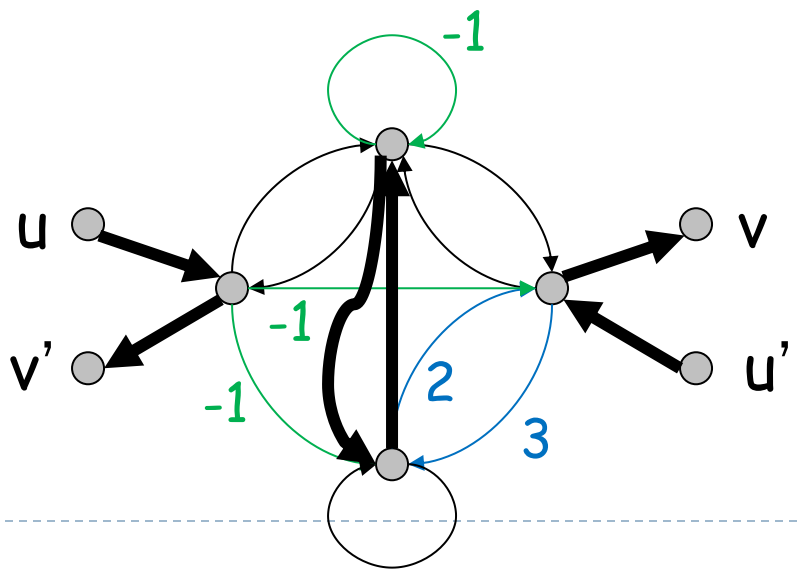
XOR gadget

- Weighted xor gadget:





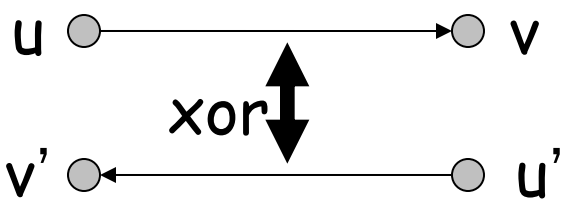


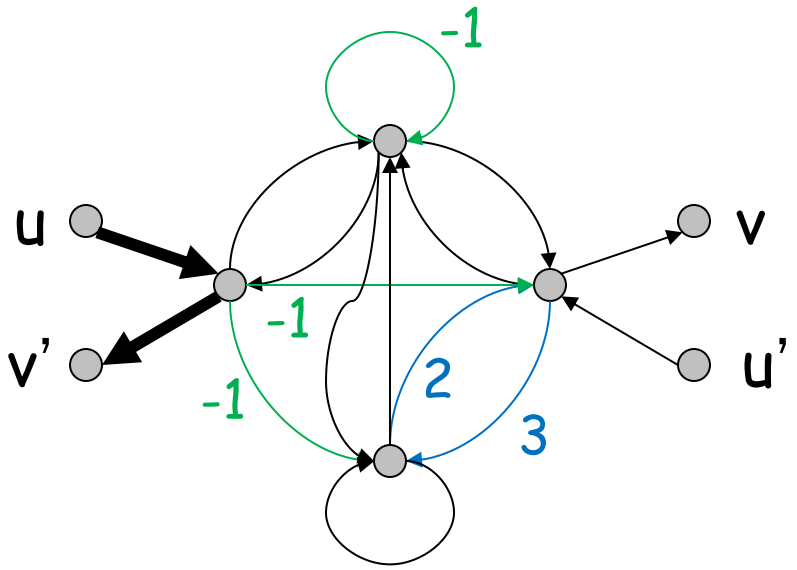




XOR gadget

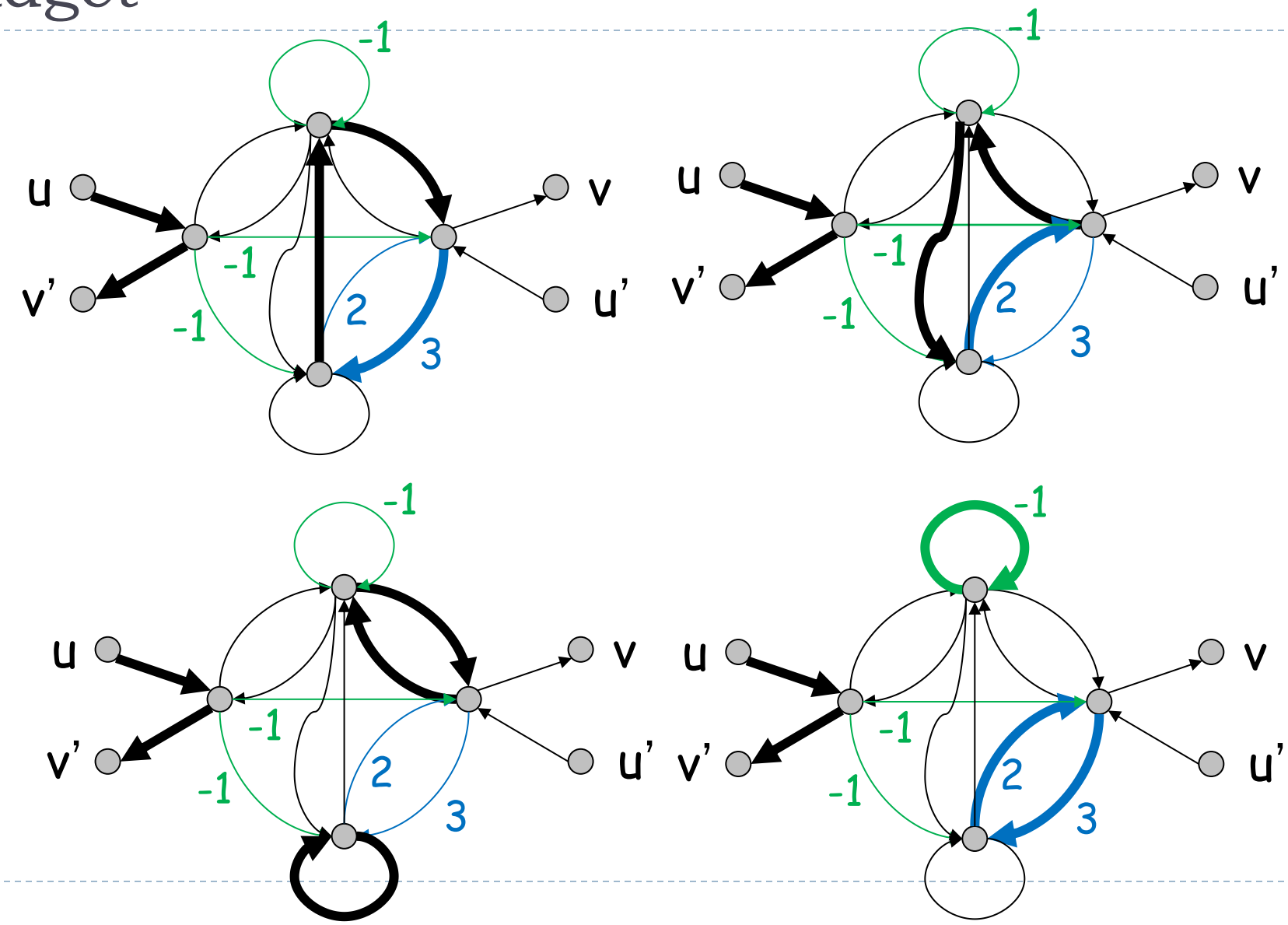
- Weighted xor gadget:



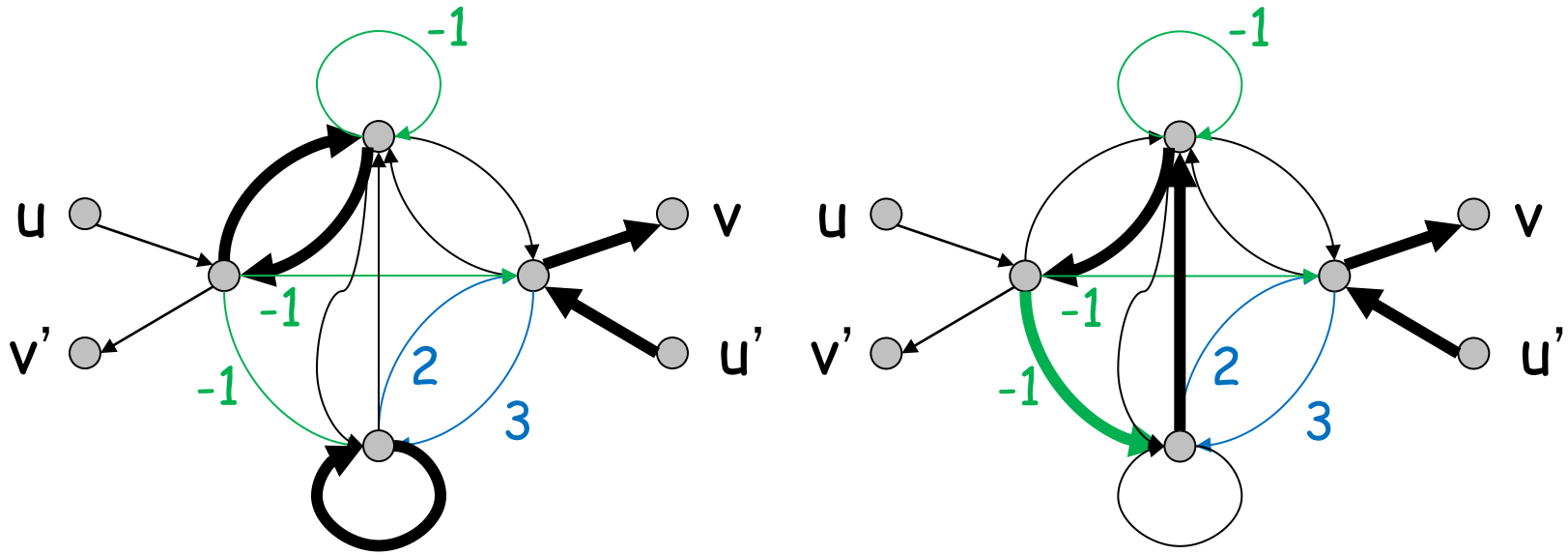




XOR gadget



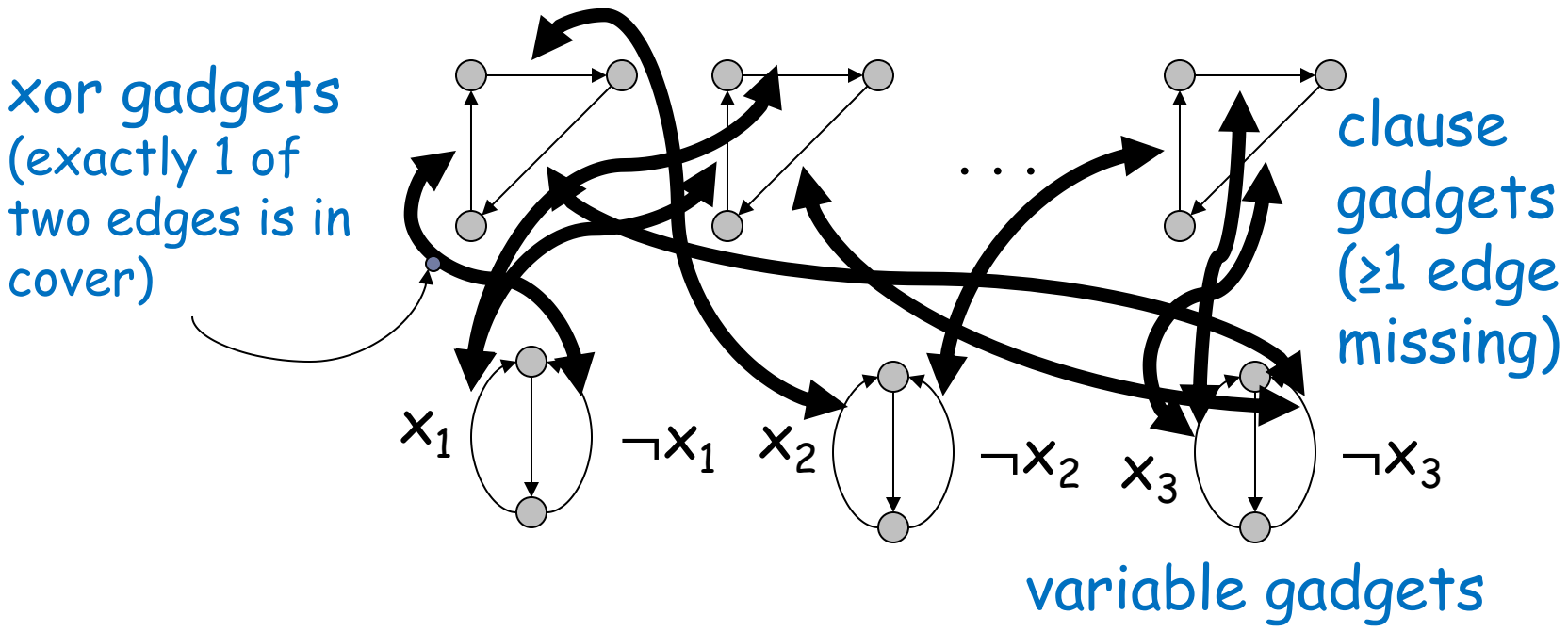
XOR gadget



Reduction

- Proof outline (reduce from #SAT)

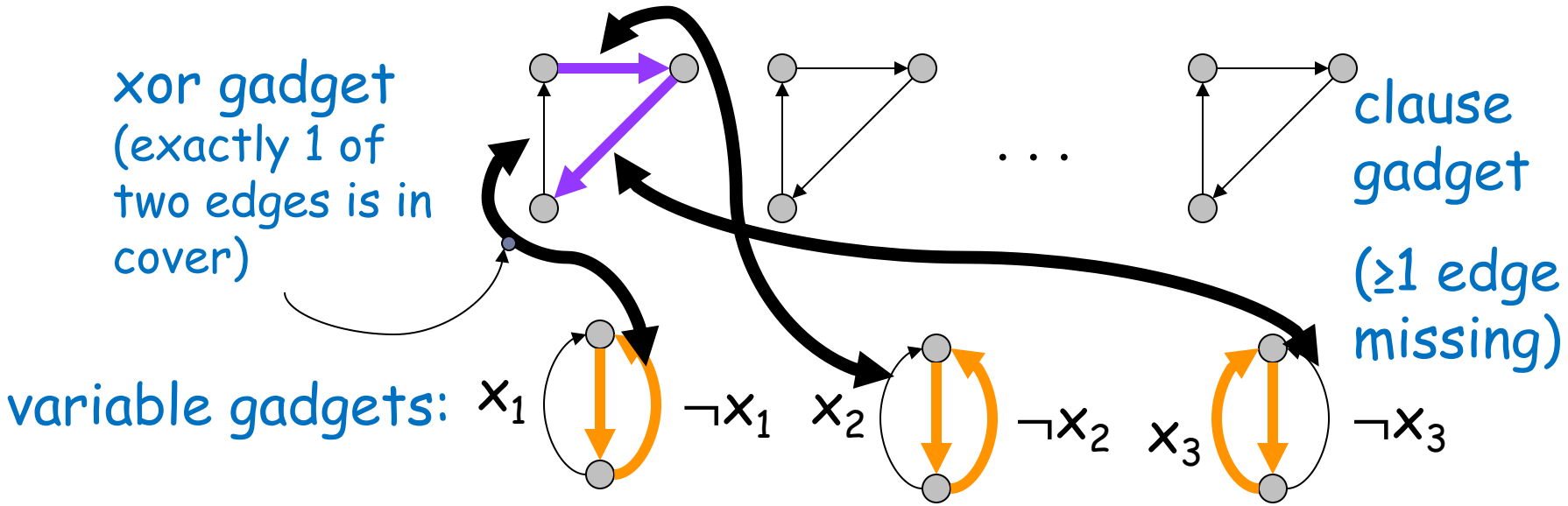
  $(\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_1) \wedge \ldots \wedge (x_3 \vee \neg x_2)$





Reduction

$(\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_3 \vee x_1) \wedge \ldots \wedge (x_3 \vee \neg x_2)$



- m = # xor gadgets; n = # variables;
- # covers = $(4^m) \cdot$ (#sat. assignments)



#CYCLE-COVER is #P-complete

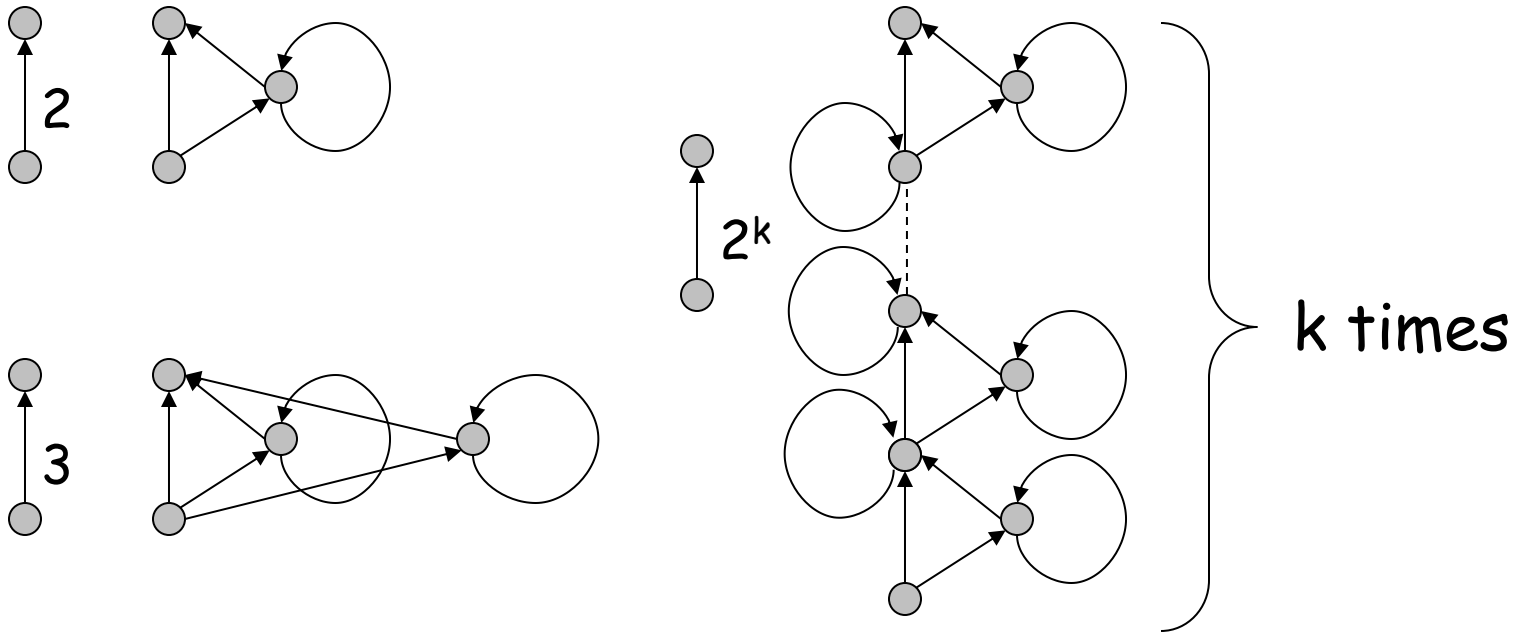
- Reduce weighted cycle cover to unweighted one?

- First reduce to positive weights
  - Pick a large $N > 4^m 2^n$
  - only consider the number module N
  - -1 becomes N-1



#CYCLE-COVER is #P-complete

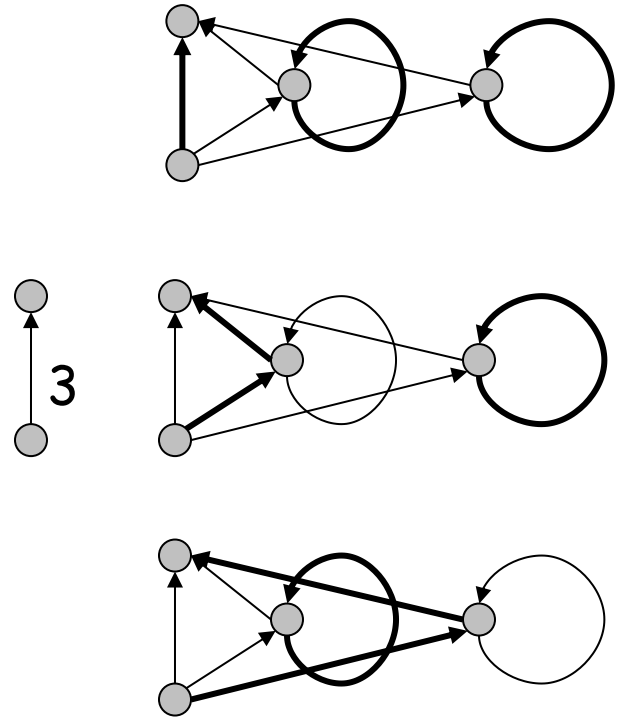
- Simulating positive edge weights
  - need to handle 2, 3, 4, 5, …, N-1

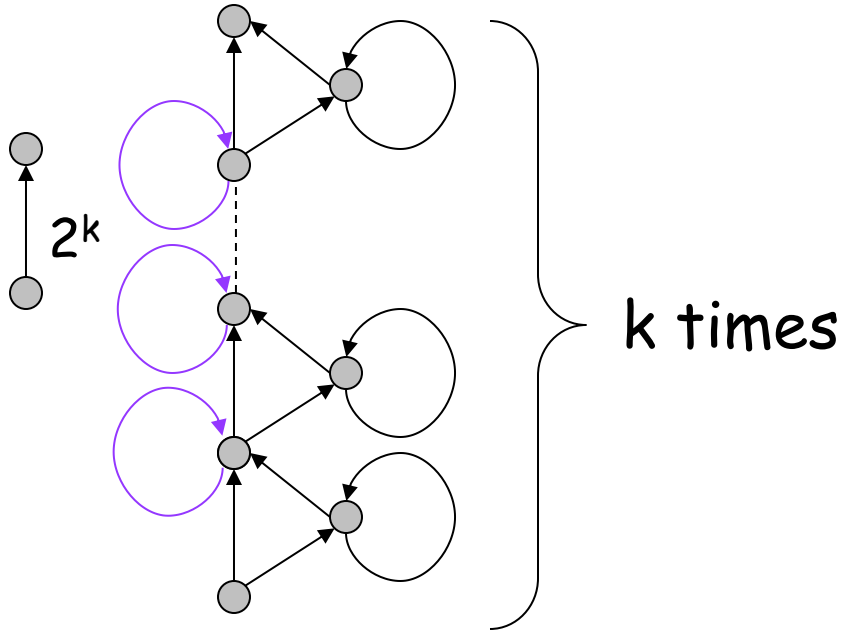




#CYCLE-COVER is #P-complete

- Simulating positive edge weights
  - need to handle 2, 3, 4, 5, …, N-1





still needs those self-loops



$\oplus$P

- PP: $x \in L \Leftrightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}| \geq \frac{1}{2} \cdot 2^{p(|x|)}$
  - "the most significant bit"

- $\oplus$P: $x \in L \Leftrightarrow |\{y \in \{0,1\}^{p(|x|)} : M(x, y)=1\}|$ is odd
  - "the least significant bit"
  - Is it easier than #P?
  - Or easier than NP?



USAT

- USAT: the language of Boolean formulas that have a unique satisfying assignment

- *[Valiant-Vazirani]* There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in USAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in SAT] = 0$

n is the number of variables

- So if $\oplus P \subseteq P$,

  $NP = RP$



USAT

- *[Valiant-Vazirani]* There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in USAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in SAT] = 0$

- Idea: random sampling
  - Construct extra part ψ for φ
  - If $2^{k-1} \leq \#\varphi \leq 2^k$, ψ has only one satisfying assignment w.h.p.
  - By pairwise independent hash functions introduced in Lecture 9



Pairwise independent hash functions

- a family of H functions $\{0,1\}^n \rightarrow \{0,1\}^k$
  - given x,y, $Pr_{h \in H}[h(x)=y]=2^{-k}$

- pairwise independent:
  - for every $x,x' \in \{0,1\}^n$ with $x \neq x'$,
  - for every $y,y' \in \{0,1\}^k$
  - $Pr_{h \in H}[h(x)=y \text{ and } h(x')=y']=2^{-2k}$

In finite field $GF(2^n)$, an example of H(x) is ax+b for $a,b \in GF(2^n)$, and we can truncate the last m-k bits to make it k bits



Statement in Lecture 9

- If we need to check $|S| \geq 2^k$ or $|S| \leq 2^{k-1}$
  - $S \subseteq \{0,1\}^n$

- Set lower bound:

Randomly pick a *pairwise independent* hash function $h:\{0,1\}^n \rightarrow \{0,1\}^{k+1}$, and randomly pick $y \in \{0,1\}^{k+1}$
When $|S| \geq 2^k$, $\Pr[\exists x \in S \text{ s.t. } h(x)=y] \geq 3/8$
When $|S| \leq 2^{k-1}$, $\Pr[\exists x \in S \text{ s.t. } h(x)=y] \leq 1/4$

So if S is larger, the prob. that such an x exists is higher



Valiant-Vazirani Lemma

- If $2^{k-1} \le |S| \le 2^k$
  - $S \subseteq \{0,1\}^n$

Randomly pick a *pairwise independent* hash function
$h:\{0,1\}^n \to \{0,1\}^{k+1}$,
if $2^{k-1} \le |S| \le 2^k$, Pr[there exists a unique $x \in S$ s.t. $h(x)=0^{k+1}$] $\ge 1/8$



Randomly pick a *pairwise independent* hash function
$h:\{0,1\}^n \rightarrow \{0,1\}^{k+1}$,
if $2^{k-1} \le |S| \le 2^k$, $Pr[\text{there exists a unique } x \in S \text{ s.t. } h(x)=0^{k+1}] \ge 1/8$

- For every $x \in S$, $Pr_h[h(x)=0^{k+1}]=2^{-(k+1)}$
- For every $x,y \in S$ and $x \neq y$, $Pr_h[h(x)=h(y)=0^{k+1}]=2^{-2(k+1)}$
- Let $N=|x \in S \text{ s.t. } h(x)=0^{k+1}|$

$$Pr_h[N \ge 1]$$
$$\ge \sum_{x \in S} Pr_h[h(x)=0^{k+1}] - \tfrac{1}{2} \sum_{x \neq y \in S} Pr_h[h(x)=h(y)=0^{k+1}]$$
$$\ge |S|/2^{k+1} - \tfrac{1}{2}|S|^2/2^{2k+2}$$

$$Pr_h[N \ge 2] \le \tfrac{1}{2}|S|^2/2^{2k+2}$$



Randomly pick a *pairwise independent* hash function
$h:\{0,1\}^n \rightarrow \{0,1\}^{k+1}$,
if $2^{k-1} \leq |S| \leq 2^k$, $Pr[\text{there exists a unique } x \in S \text{ s.t. } h(x)=0^{k+1}] \geq 1/8$

- For every $x \in S$, $Pr_h[h(x)=0^{k+1}]=2^{-(k+1)}$
- For every $x,y \in S$ and $x \neq y$, $Pr_h[h(x)=h(y)=0^{k+1}]=2^{-2(k+1)}$
- Let $N=|x \in S \text{ s.t. } h(x)=0^{k+1}|$

$$Pr_h[N \geq 1] \geq |S|/2^{k+1} - \tfrac{1}{2}|S|^2/2^{2k+2}$$

$$Pr_h[N \geq 2] \leq \tfrac{1}{2}|S|^2/2^{2k+2}$$

$$Pr_h[N=1] = Pr_h[N \geq 1] - Pr_h[N \geq 2] \geq |S|/2^{k+1} - |S|^2/2^{2k+2}$$

$$\geq 3/16 > 1/8$$

(since $1/4 \leq |S|/2^{k+1} \leq 1/2$)



Randomly pick a *pairwise independent* hash function
$h:\{0,1\}^n \to \{0,1\}^{k+1}$,
if $2^{k-1} \le |S| \le 2^k$, Pr[there exists a unique $x \in S$ s.t. $h(x)=0^{k+1}$] $\ge 1/8$

- Given a Boolean formula φ,
  - pick k randomly from {1,…,n}
  - pick $h:\{0,1\}^m \to \{0,1\}^{k+1}$ randomly
  - consider the formula $\psi(x) = \varphi(x) \wedge (h(x)=0^{k+1})$

  - With prob. 1/n, we can choose the correct k s.t. $2^{k-1} \le |S| \le 2^k$ where $S=\{x: \varphi(x)=1\}$
  - So when $\varphi \in$ SAT, with prob. $1/(8n)$, $\psi \in$ USAT

  - *(For more details see the textbook)*



USAT

- USAT: the language of Boolean formulas that have a unique satisfying assignment

- *[Valiant-Vazirani]* There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in USAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in SAT] = 0$

Hard to boost this prob. for USAT



USAT and $\oplus$SAT

- $\oplus$SAT: the language of Boolean formulas that have an odd number of satisfying assignments

- There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] = 0$

We can boost this prob.



Reducing error prob.

- There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] = 0$

- We run for k times and get $\psi_1, \ldots, \psi_K$
  - if one of $\#\psi_i$ is odd, $\varphi \in SAT$

- Construct a formula ψ such that
  - $\#\psi$ is odd $\Leftrightarrow$ one of $\#\psi_i$ is odd



Construction of new formulas

- Given two formulas α and β on n variables, how to construct a new formula γ such that:
  - $\#\gamma=\#\alpha\cdot\#\beta$
  - $\#\gamma=\#\alpha+\#\beta$



Construction of new formulas

- Given two formula α and β on n variables, how to construct a new formula γ such that:
  - $\#\gamma=\#\alpha\cdot\#\beta$

    $\gamma(x,y)=\alpha(x)\wedge\beta(y)$ (γ has 2n variables)
  - $\#\gamma=\#\alpha+\#\beta$

    $\gamma(x)=((x_0=0)\wedge\alpha(x_1,\ldots,x_n))\vee((x_0=1)\wedge\beta(x_1,\ldots,x_n))$

    (γ has n+1 variables)



Construction of new formulas

- Given two formula α and β on n variables, how to construct a new formula γ such that:
  - $\#\gamma=\#\alpha\cdot\#\beta$

    $\gamma(x,y)=\alpha(x)\wedge\beta(y)$ (γ has 2n variables)
  - $\#\gamma=\#\alpha+\#\beta$

    $\gamma(x)=((x_0=0)\wedge\alpha(x_1,\ldots,x_n))\vee((x_0=1)\wedge\beta(x_1,\ldots,x_n))$

    (γ has n+1 variables)
  - $\#\gamma=\#\alpha+1$

    make β only have 1 satisfying assignment



Reducing error prob.

- There is a p.p.t. algorithm f such that for every φ:

  $\varphi \in SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow Pr[f(\varphi) \in \oplus SAT] = 0$

- We run for k times and get $\psi_1,\ldots,\psi_K$, construct a formula ψ:

  $\#\psi=(\#\psi_1+1)(\#\psi_2+1)\ \ldots(\#\psi_K+1)+1$

- So $\#\psi$ is odd $\Leftrightarrow$ one of $\#\psi_i$ is odd
- So we can reduce the error prob. to 2^{-m} by K=poly(n,m)

- So if $\oplus P \subseteq P$, RP=NP



Logic operation on $\oplus$ operator

- $(\oplus_x \alpha(x)) \wedge (\oplus_y \beta(y)) \Leftrightarrow \quad \oplus_{x,y}(\alpha(x) \wedge \beta(y))$
- $\sim \oplus_x \alpha(x) \quad \Leftrightarrow \quad \oplus_{x,z}((z_0=0) \wedge \alpha(x) \vee ((z_0=1) \wedge (x=0))$
- $(\oplus_x \alpha(x)) \vee (\oplus_y \beta(y)) \Leftrightarrow$ (By De Morgan's rule)
- Any formula α (with given assignment) can be seen $\oplus \alpha$
  - equals 1 if α is true
  - equals 0 otherwise



What about…

- $\oplus_z((\oplus_x \alpha(x, z)) \wedge (\oplus_y \beta(y, z)))$

$\Leftrightarrow \oplus_z(\oplus_{x,y}(\alpha(x, z) \wedge \beta(y, z)))$

$\Leftrightarrow \oplus_{x,y,z}(\alpha(x, z) \wedge \beta(y, z))$



Valiant-Vazirani

- For a random pairwise hash function h and random k, for every φ:

  $\varphi \in SAT \Rightarrow \Pr[\ \varphi(x) \wedge (h(x)=0^{k+1}) \in \bigoplus SAT] \geq 1/(8n)$

  $\varphi \notin SAT \Rightarrow \Pr[\ \varphi(x) \wedge (h(x)=0^{k+1}) \in \bigoplus SAT] = 0$

- In fact, that holds for every Boolean function φ



Valiant-Vazirani

- There is a probabilistic polynomial-time procedure that, given input 1^n, produces a formula $\tau(x)$ such that for every Boolean function φ:

$$\exists x(\varphi(x)) \quad \Rightarrow \Pr[\oplus(\varphi(x) \wedge \tau(x))] \geq 1/(8n)$$

$$\sim\exists x(\varphi(x)) \quad \Rightarrow \Pr[\oplus(\varphi(x) \wedge \tau(x))] = 0$$

- So by error reduction:

$$\forall x(\varphi(x))$$

$$\Updownarrow \text{ w.h.p.}$$

$$(\sim\oplus(\sim\varphi(x) \wedge \tau_1(x))) \wedge (\sim\oplus(\sim\varphi(x) \wedge \tau_2(x))) \ldots \wedge (\sim\oplus(\sim\varphi(x) \wedge \tau_K(x)))$$

$$\Updownarrow$$

$$\oplus(\alpha(x))$$



Power of $\oplus$P

$\exists y \forall x(\varphi(x, y))$



w.h.p.

$\exists y(\oplus_x(\alpha(x, y)))$



w.h.p.

$\oplus_{x,y}(\beta(x, y))$



Randomized reduction from **PH** to $\oplus$SAT

- There exists a p.p.t. algorithm A that for any quantified Boolean formula ψ of size n with constant levels of alternations, and for any m, runs in poly(n,m) time, s.t:

  $\psi \Longrightarrow \Pr[A(\psi) \in \oplus SAT] \geq 1\text{-}2^{-m}$

  $\sim\psi \Longrightarrow \Pr[A(\psi) \in \oplus SAT] \leq 2^{-m}$

- Thus, **PH** $\subseteq$ **BPP**$^{\oplus \mathbf{P}}$



Derandomization

- The p.p.t function $A(\psi)$ can be seen as a deterministic function $A(\psi,r)$ with random vector r

- Let m=2, then r has k=poly(n) bits, and:

$$\psi \Longrightarrow \Pr_r[A(\psi,r) \in \oplus SAT] \geq 3/4$$

$$\sim\psi \Longrightarrow \Pr_r[A(\psi,r) \in \oplus SAT] \leq 1/4$$

- We want to make it deterministic by counting



Derandomization (by #P oracle)

- There is a deterministic polynomial-time function T that, for every Boolean formula α, outputs a formula $\beta = T(\alpha, 1^\ell)$ such that

$$\alpha \in \oplus\text{SAT} \Longrightarrow \#\beta \equiv -1 \pmod{2^{\ell+1}}$$

$$\alpha \notin \oplus\text{SAT} \Longrightarrow \#\beta \equiv 0 \pmod{2^{\ell+1}}$$



- There is a deterministic polynomial-time function T that, for every Boolean formula α, outputs a formula $\beta = T(\alpha, 1^\ell)$ such that

$$\alpha \in \oplus\text{SAT} \Longrightarrow \#\beta \equiv -1 \pmod{2^{\ell+1}}$$

$$\alpha \notin \oplus\text{SAT} \Longrightarrow \#\beta \equiv 0 \pmod{2^{\ell+1}}$$

- Recall we can make new formula γ such that:

$$\#\gamma = \#\alpha \cdot \#\beta$$

$$\gamma(x,y) = \alpha(x) \wedge \beta(y)$$

$$\#\gamma = \#\alpha + \#\beta$$

$$\gamma(x) = ((x_0 = 0) \wedge \alpha(x_1, \ldots, x_n)) \vee ((x_0 = 1) \wedge \beta(x_1, \ldots, x_n))$$

- So we can make γ so that $\#\gamma = 4(\#\alpha)^3 + 3(\#\alpha)^4$, and the length only increase by O(1) factor



- There is a deterministic polynomial-time function T that, for every Boolean formula α, outputs a formula $\beta = T(\alpha, 1^{\ell})$ such that

$$\alpha \in \oplus\mathrm{SAT} \Longrightarrow \#\beta \equiv -1 \pmod{2^{\ell+1}}$$

$$\alpha \notin \oplus\mathrm{SAT} \Longrightarrow \#\beta \equiv 0 \pmod{2^{\ell+1}}$$

- So we can make γ so that $\#\gamma = 4(\#\alpha)^3 + 3(\#\alpha)^4$, and the length only increase by O(1) factor

- You can check:

$$x \equiv -1 \pmod{a} \Longrightarrow 4x^3 + 3x^4 \equiv -1 \pmod{a^2}$$

$$x \equiv 0 \pmod{a} \Longrightarrow 4x^3 + 3x^4 \equiv 0 \pmod{a^2}$$

- So by $O(\log \ell)$ times we can make $\#\beta \equiv -1$ or 0 module $2^{\ell+1}$, and the length of β is $2^{O(\log \ell)}$ times α



Derandomization

$\alpha \in \oplus SAT \Rightarrow \#\beta \equiv -1 \pmod{2^{\ell+1}}$
$\alpha \notin \oplus SAT \Rightarrow \#\beta \equiv 0 \pmod{2^{\ell+1}}$

- The p.p.t function $A(\psi)$ can be seen as a deterministic function $A(\psi,r)$ with random vector r.
- Let m=2, then r has k=poly(n) bits, and:

$$\psi \Rightarrow Pr_r[A(\psi,r) \in \oplus SAT] \geq 3/4$$
$$\sim\psi \Rightarrow Pr_r[A(\psi,r) \in \oplus SAT] \leq 1/4$$

- Make ℓ=k+2, consider $\beta' = T(A(\psi,r), 1^{\ell})$, then:
  - If ψ is true, $\geq \frac{3}{4}2^k$ different r's make $A(\psi,r) \in \oplus SAT$, so $\#\beta'$ module $2^{\ell+1}$ is between $\left[-2^k, -\frac{3}{4}2^k\right]$
  - If ψ is false, $\leq \frac{1}{4}2^k$ different r's make $A(\psi,r) \in \oplus SAT$, so $\#\beta'$ module $2^{\ell+1}$ is between $\left[-\frac{1}{4}2^k, 0\right]$
  - We can see these two ranges are disjoint, so by a $\#\beta'$ oracle we can answer ψ

Here we count all possible r's



Toda's theorem

- **PH ⊆ P^#P**

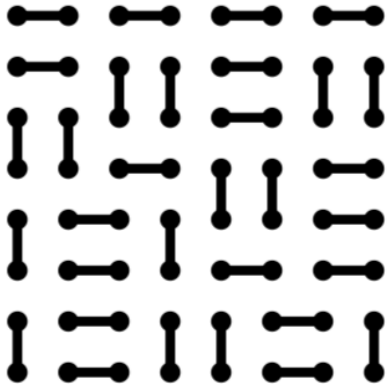
- Of course if **#P=FP**, then **P=NP** and **P=PH**

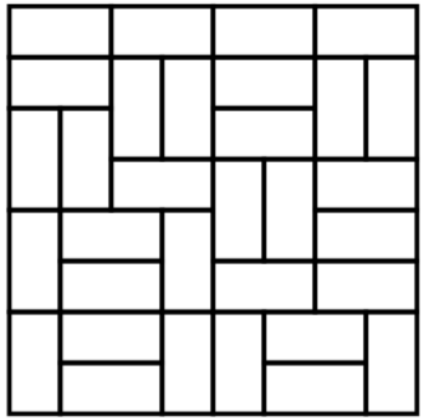
- But Toda's theorem means more:
  - If we have an $n^{O(\log n)}$-time algorithm for #SAT, then we also have such an algorithm for any problem in **PH**
  - But it's not know whether an $n^{O(\log n)}$-time algorithm for SAT will imply this



#matchings

- Counting perfect matchings in general graphs is #P-complete
- Counting perfect matchings in planar graphs is in polynomial time (FKT algorithm)
  - example: domino tiling





- Counting matchings in planar graph is also #P-complete *[Jerrum 1987]*



Determinant

- Definition of determinant:

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=0}^{n-1} A_{i,\sigma(i)}$$

  - σ ranges over all permutations of {0,…,n-1}
  - $\operatorname{sgn}(\sigma)$=+1 if σ is an even permutation
    $\operatorname{sgn}(\sigma)$=-1 if σ is an odd permutation

A_{00}	A_{01}	A_{02}	A_{03}
A_{10}	A_{11}	A_{12}	A_{13}
A_{20}	A_{21}	A_{22}	A_{23}
A_{30}	A_{31}	A_{32}	A_{33}

σ= (1,2,0,3)



Parity of permutation

- $N(\sigma)$ of a permutation σ is the number of inversions, that is, x<y and $\sigma(x)>\sigma(y)$
  - Even permutation σ: $N(\sigma)$ is even
  - Odd permutation σ: $N(\sigma)$ is odd

x	1	2	3	4	5
$\sigma(x)$	3	4	5	2	1



Parity of permutation

- $N(\sigma)$ of a permutation σ is the number of inversions, that is, $x<y$ and $\sigma(x)>\sigma(y)$
  - Even permutation σ: $N(\sigma)$ is even
  - Odd permutation σ: $N(\sigma)$ is odd
- Another definition: Even/odd number of transitions (exchanges) from the identity permutation
  - Many ways, but the parity is the same

x	1	2	3	4	5
$\sigma(x)$	3	4	5	2	1

1 2 3 4 5
⇒ 1 4 3 2 5
⇒ 1 4 5 2 3
⇒ 3 4 5 2 1



Composition of permutation

- Composition of σ and π: $(\pi \circ \sigma)(x)=\pi(\sigma(x))$

x	1	2	3	4	5
$\sigma(x)$	3	4	5	2	1

x	1	2	3	4	5
$\pi(x)$	4	1	2	5	3

$\pi \circ \sigma$	2	5	3	1	4

- the composition of two even permutations is even
- the composition of two odd permutations is even
- the composition of an odd and an even permutation is odd

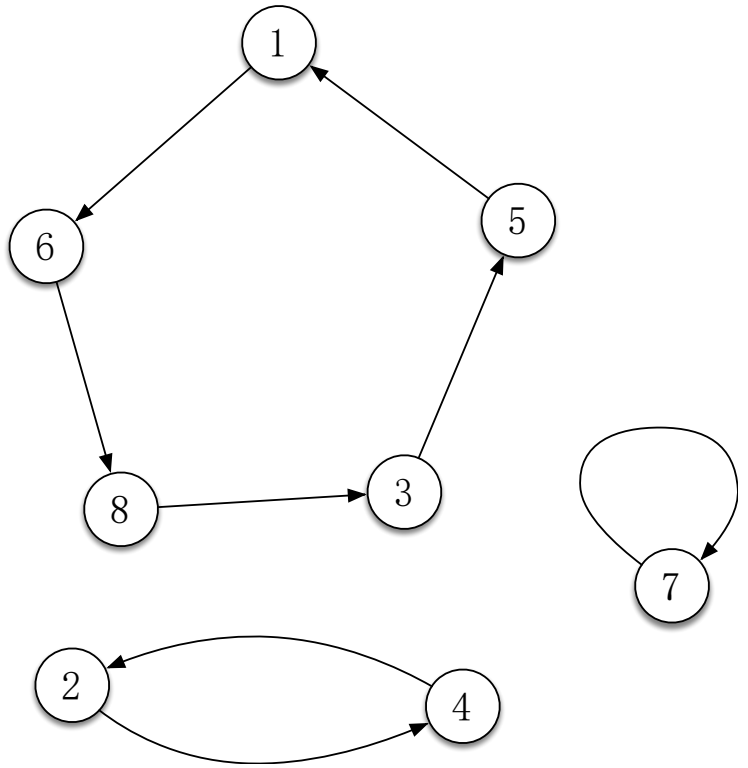


Cycles in permutation

- Cycle: x, $\sigma(x)$, $\sigma(\sigma(x))$, $\sigma(\sigma(\sigma(x)))$,…, until back to x

- Parity of even-length cycle?
- Parity of odd-length cycle?

1	2	3	4	5	6	7	8
6	4	5	2	1	8	7	3



Every node has one incoming edge and one outgoing edge, so it will form disjoint cycles

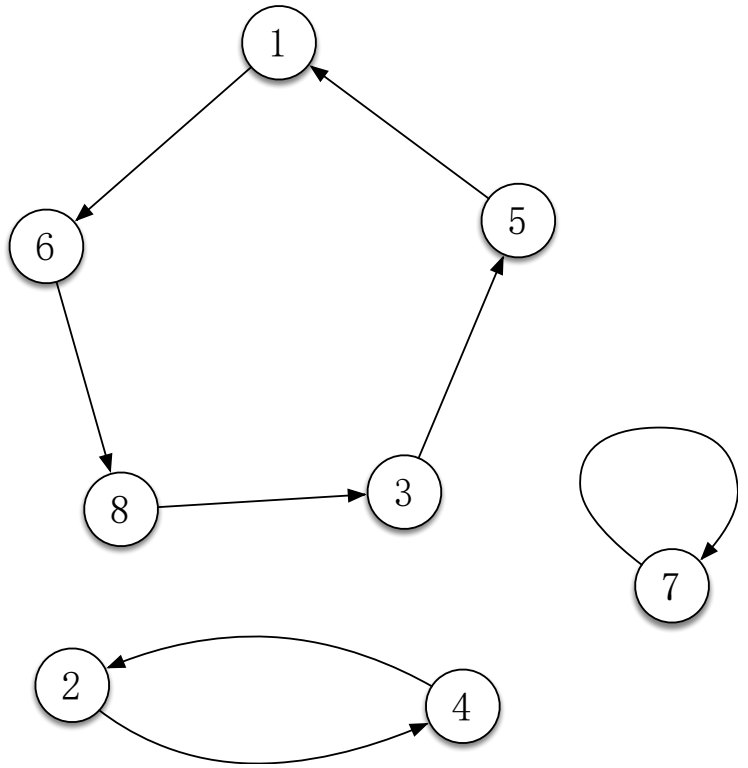


Cycles in permutation

- Cycle: x, $\sigma(x)$, $\sigma(\sigma(x))$, $\sigma(\sigma(\sigma(x)))$,…, until back to x

- Parity of even-length cycle: odd
- Parity of odd-length cycle: even
  - Move a_1 to front in $a_2, a_3, a_4, \ldots, a_n, a_1$: n-1 exchanges

1	2	3	4	5	6	7	8
6	4	5	2	1	8	7	3



Every node has one incoming edge and one outgoing edge, so it will form disjoint cycles

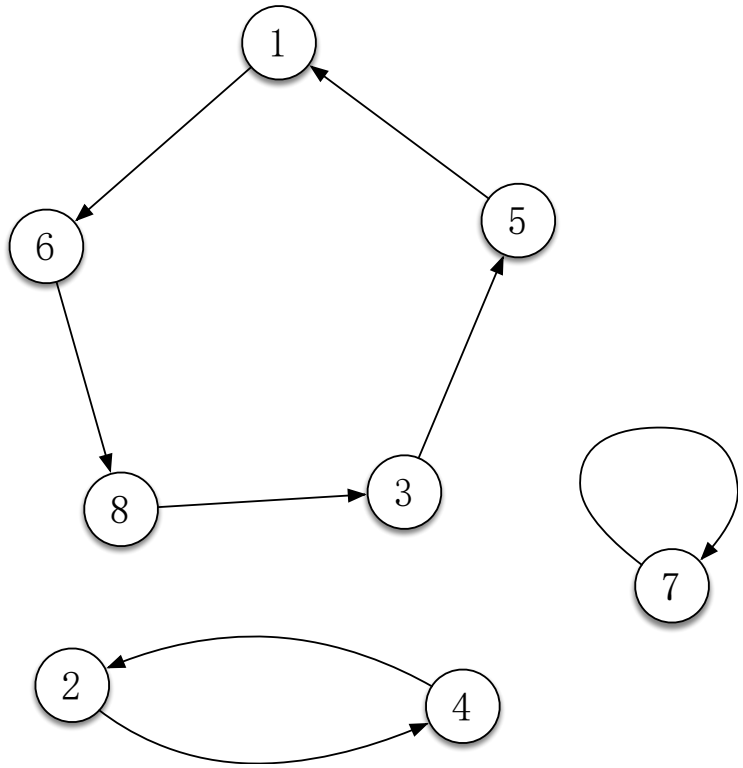


Cycles in permutation

- Cycle: x, $\sigma(x)$, $\sigma(\sigma(x))$, $\sigma(\sigma(\sigma(x)))$,…, until back to x

- Parity of even-length cycle: odd
- Parity of odd-length cycle: even

- Permutation: composition of cycles
- Easy way to compute parity:
  - Odd permutation ⇔ odd number of even-length cycles



Odd permutation



Skew-symmetric matrix

- an $n \times n$ matrix A is skew-symmetric if $A^T=-A$
  - $A_{i,j}=-A_{j,i}$
  - $A_{i,i}=0$

- Consider det(A):
  - $\det(A)=\det(-A^T)=(-1)^n\det(A)$
  - So if n is odd, det(A)=0

0	$A_{1,2}$	$A_{1,3}$	$A_{1,4}$
$-A_{1,2}$	0	$A_{2,3}$	$A_{2,4}$
$-A_{1,3}$	$-A_{2,3}$	0	$A_{3,4}$
$-A_{1,4}$	$-A_{2,4}$	$-A_{3,4}$	0

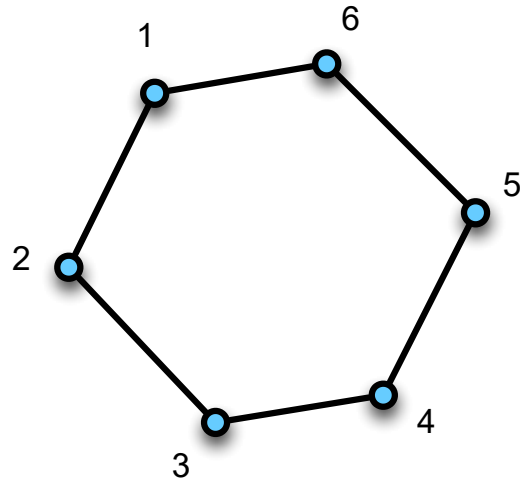


Tutte matrix and general graph matching

- Tutte matrix T of G=(V, E):

$$T_{ij} = \begin{cases} 0 & \text{if } \{i,j\} \notin E \\ x_{ij} & \text{if } \{i,j\} \in E \text{ and } i < j \\ -x_{ji} & \text{if } \{i,j\} \in E \text{ and } i > j \end{cases}$$

0	x_{12}	0	0	0	x_{16}
$-x_{12}$	0	x_{23}	0	0	0
0	$-x_{23}$	0	x_{34}	0	0
0	0	$-x_{34}$	0	x_{45}	0
0	0	0	$-x_{45}$	0	x_{56}
$-x_{16}$	0	0	0	$-x_{56}$	0





Skew-symmetric matrix

- an $n \times n$ matrix A is skew-symmetric if $A^T=-A$
  - $A_{i,j}=-A_{j,i}$
  - $A_{i,i}=0$

- Only consider det(A) for even n
  - Related to perfect matchings

0	$A_{1,2}$	$A_{1,3}$	$A_{1,4}$
$-A_{1,2}$	0	$A_{2,3}$	$A_{2,4}$
$-A_{1,3}$	$-A_{2,3}$	0	$A_{3,4}$
$-A_{1,4}$	$-A_{2,4}$	$-A_{3,4}$	0

$$\det(A)=(A_{1,2}A_{3,4}-A_{1,3}A_{2,4}+A_{1,4}A_{2,3})^2$$

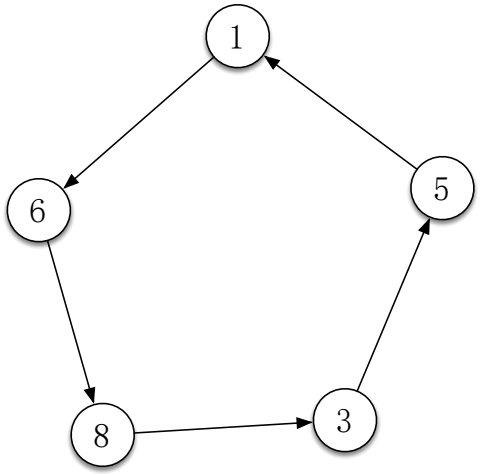


Determinant of skew-symmetric matrix

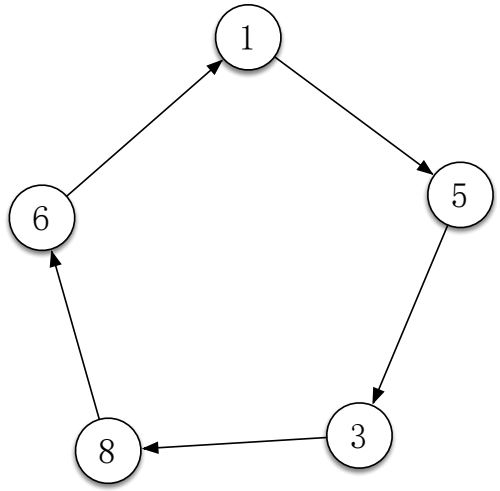
- $\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=0}^{n-1} A_{i,\sigma(i)}$
- Consider the cycles of σ:
  - Self-cycle: if σ(i)=i for some i, $A_{i,i}=0$
  - Odd-cycles:

    - Two permutations σ,σ' only differ by reversing the smallest-indexed odd cycle
    - $\operatorname{sgn}(\sigma)=\operatorname{sgn}(\sigma')$, $\prod_{i=0}^{n-1} A_{i,\sigma(i)} = -\prod_{i=0}^{n-1} A_{i,\sigma'(i)}$
    - So they cancel each other

- Thus, only consider σ consisting of even cycles



$A_{1,6}A_{6,8}A_{8,3}A_{3,5}A_{5,1}$

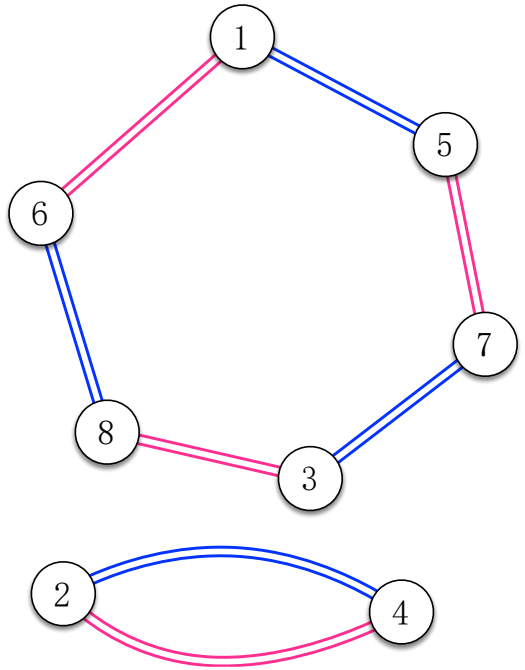


$A_{6,1}A_{8,6}A_{3,8}A_{5,3}A_{1,5}$=-$A_{1,6}A_{6,8}A_{8,3}A_{3,5}A_{5,1}$

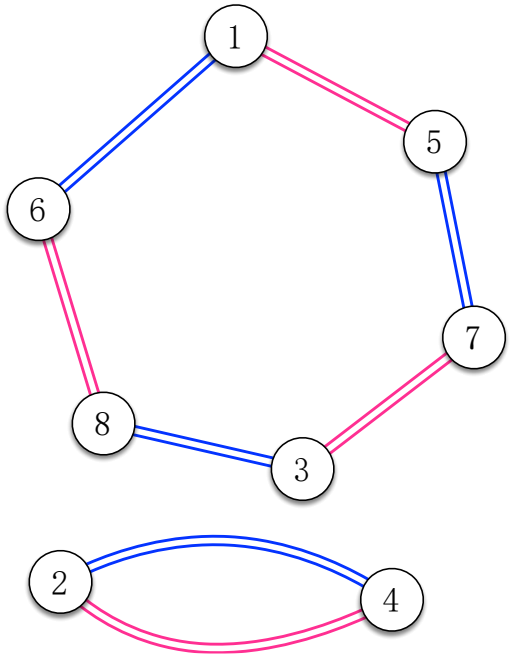


Even cycles and perfect matchings

- We know that two perfect matchings can form even cycles:



M_1, M_2

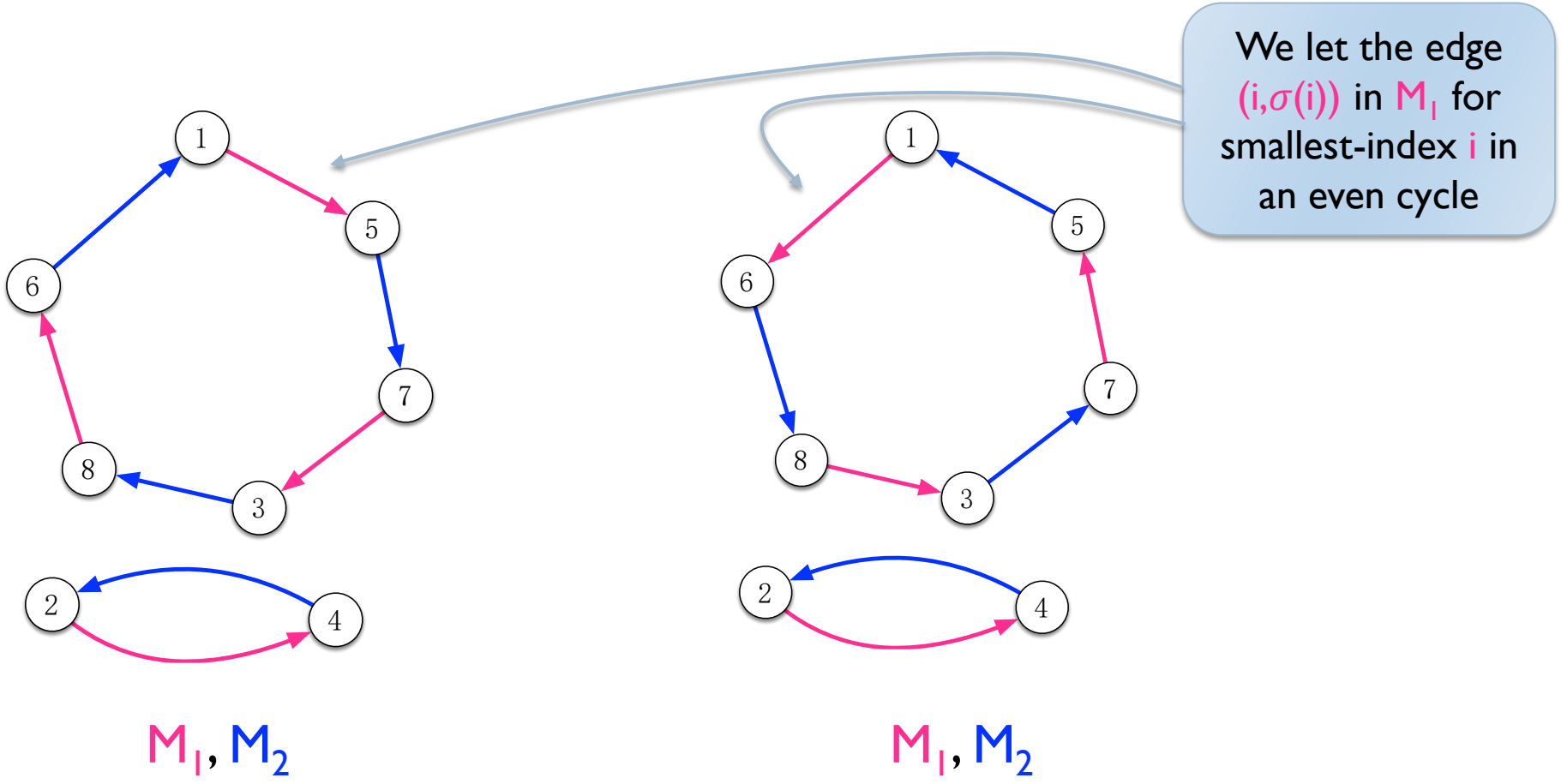


M_1, M_2



Even cycles and perfect matchings

- We know that two perfect matchings can form even cycles:

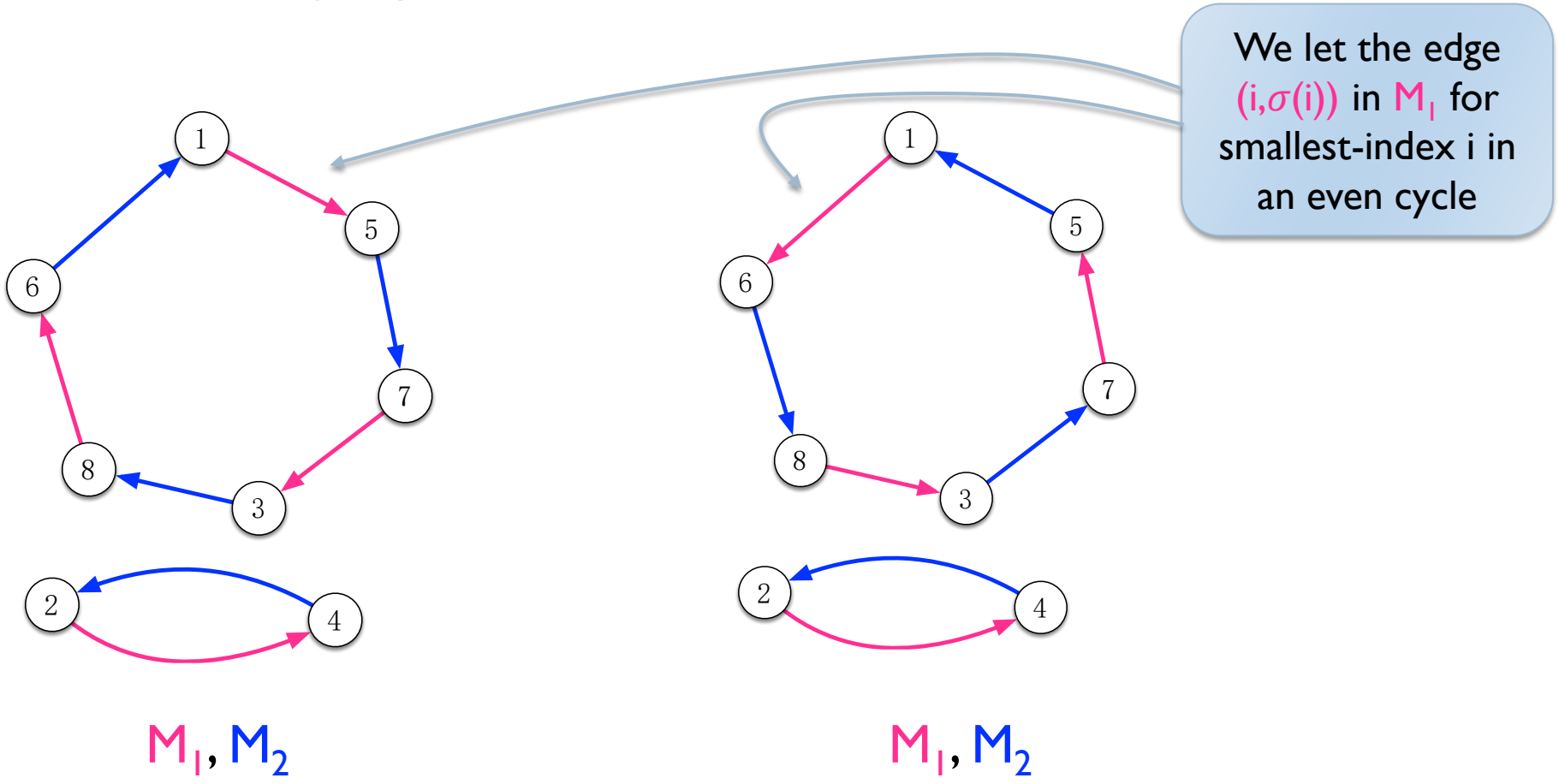


- One-to-One mapping:

  Pair of perfect matchings M_1, M_2 in complete graph K_n

  $\Leftrightarrow$ Even-length directed-cycle-partitions of nodes {1,…,n}

  $\Leftrightarrow$ Permutation σ of even-length cycles



Weight of matching in skew-symmetric matrix

- a permutation σ associated with a matching M:

  M={(σ(2i-1),σ(2i)): 1≤i≤n/2}

M={(1,4),(3,6),(2,5)}
(1,4,3,6,2,5), (6,3,5,2,1,4), (2,5,4,1,3,6) are all associated with M

- Define $\mathrm{wt}(\mathrm{M}) = \mathrm{sgn}(\sigma)\prod_{i=1}^{n/2} A_{\sigma(2i-1),\sigma(2i)}$
- (It is the same for different σ associated with M)

Exchange two matching edges:
(1,4,5,2,3,6) ⇒ (3,6,5,2,1,4)
$\prod_{i=1}^{n/2} A_{\sigma(2i-1),\sigma(2i)}$ doesn't change
two transitions, sgn(σ) will not change

Switch two vertices of a matching edge:
(1,4,5,2,3,6) ⇒ (1,4,2,5,3,6)
$A_{\sigma(2i-1),\sigma(2i)} = -A_{\sigma(2i),\sigma(2i-1)}$
one transition, sgn(σ) also times -1



Pfaffian of skew-symmetric matrix

- a permutation σ associated with a matching M: M={(σ(2i-1),σ(2i)): 1≤i≤n/2}
- Define $\mathrm{wt}(\mathrm{M}) = \mathrm{sgn}(\sigma)\prod_{i=1}^{n/2} A_{\sigma(2i-1),\sigma(2i)}$
- $\mathcal{M}_n$ is the set of all perfect matchings in complete graph K_n

- Let A be an n×n skew-symmetric matrix with n even, the Pfaffian of A:

$$\mathrm{Pf}(A) = \sum_{M \in \mathcal{M}_n} \mathrm{wt}(M)$$

- THM *(Cayley 1842)* det(A)=(Pf(A))^2



THM *(Cayley 1842)* $\det(A)=(\mathrm{Pf}(A))^2$

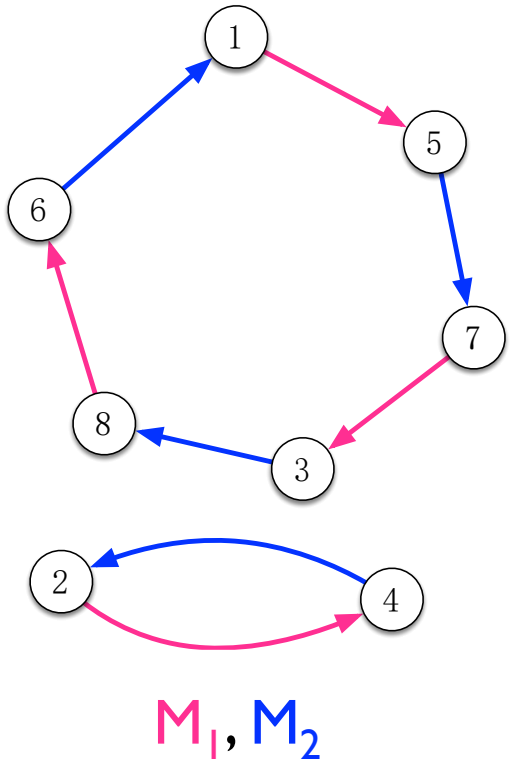
$$(\mathrm{Pf}(A))^2 = \sum_{(M,M')\in\mathcal{M}_n\times\mathcal{M}_n} \mathrm{wt}(M)\cdot\mathrm{wt}(M')$$

- Since A is skew-symmetric, only terms with σ consisting of even cycles are non-zero:

$$\det(A) = \sum_{\sigma \text{ consisting of even cycles}} \mathrm{sgn}(\sigma)\prod_{i=0}^{n-1} A_{i,\sigma(i)}$$

- Such σ and (M,M') are one-to-one correspondence:

x	1	2	3	4	5	6	7	8
σ(x)	5	4	8	2	7	1	3	6



M_1, M_2

THM *(Cayley 1842)* $\det(A)=(\mathrm{Pf}(A))^2$

$$(\mathrm{Pf}(A))^2 = \sum_{(M,M')\in\mathcal{M}_n\times\mathcal{M}_n} \mathrm{wt}(M)\cdot\mathrm{wt}(M')$$

- α, α' associated with M, M', resp.

$$\mathrm{wt}(M) = \mathrm{sgn}(\alpha)\prod_{i=1}^{n/2} A_{\alpha(2i-1),\alpha(2i)}$$

$$\mathrm{wt}(M') = \mathrm{sgn}(\alpha')\prod_{i=1}^{n/2} A_{\alpha'(2i-1),\alpha'(2i)}$$

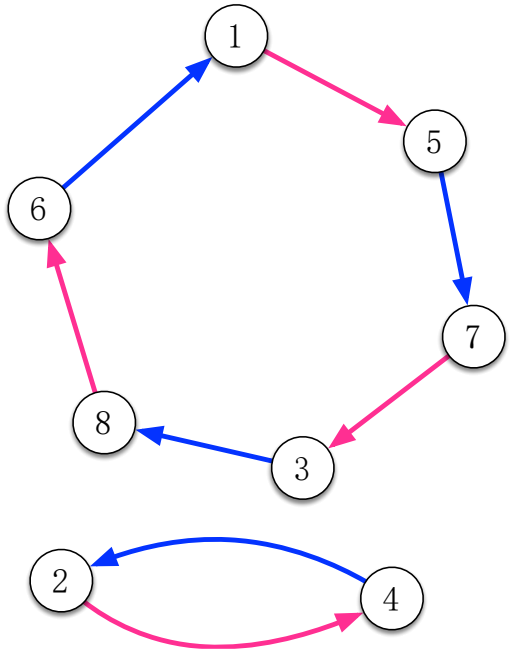
- $\prod A_{i,\sigma(i)}$ terms are the same

- Since A is skew-symmetric, only terms with σ consisting of even cycles are non-zero:

$$\det(A) = \sum_{\sigma \text{ consisting of even cycles}} \mathrm{sgn}(\sigma)\prod_{i=0}^{n-1} A_{i,\sigma(i)}$$

- Such σ and (M,M') are one-to-one correspondence:

x	1	2	3	4	5	6	7	8
σ(x)	5	4	8	2	7	1	3	6



M_1, M_2

THM *(Cayley 1842)* $\det(A)=(\mathrm{Pf}(A))^2$

$$(\mathrm{Pf}(A))^2 = \sum_{(M,M')\in\mathcal{M}_n\times\mathcal{M}_n} \mathrm{wt}(M)\cdot\mathrm{wt}(M')$$

- α, α' associated with M, M', resp.

$$\mathrm{wt}(M) = \mathrm{sgn}(\alpha)\prod_{i=1}^{n/2} A_{\alpha(2i-1),\alpha(2i)}$$

$$\mathrm{wt}(M') = \mathrm{sgn}(\alpha')\prod_{i=1}^{n/2} A_{\alpha'(2i-1),\alpha'(2i)}$$

- $\Pi A_{i,\sigma(i)}$ terms are the same

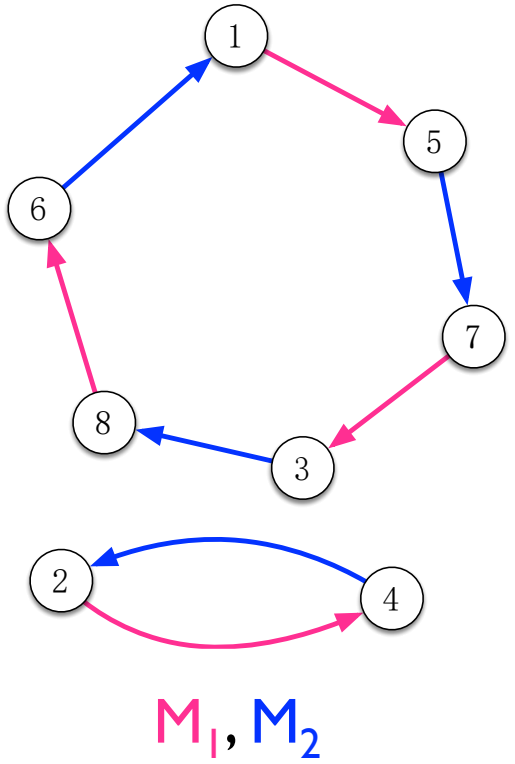
- Since A is skew-symmetric, only terms with σ consisting of even cycles are non-zero:

$$\det(A) = \sum_{\sigma \text{ consisting of even cycles}} \mathrm{sgn}(\sigma)\prod_{i=0}^{n-1} A_{i,\sigma(i)}$$

- S

$\mathrm{sgn}(\sigma)=(-1)^k$, where k is the number even cycles in σ
If M, M' forms one even-length cycle, $\mathrm{sgn}(\alpha)\cdot\mathrm{sgn}(\alpha')=-1$
- choose α, α' by the order of the cycles:
- e.g. $\alpha=(1,5,7,3,8,6)$, $\alpha'=(5,7,3,8,6,1)$, odd number of transitions

So $\mathrm{sgn}(\alpha)\cdot\mathrm{sgn}(\alpha')=(-1)^k=\mathrm{sgn}(\sigma)$



M_1, M_2

Counting perfect matching

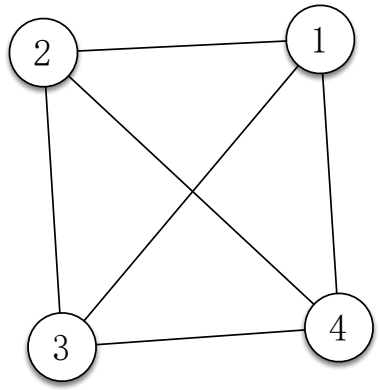
- THM *(Cayley 1842)* det(A)=(Pf(A))^2, where $\mathrm{Pf}(A) = \sum_{M \in \mathcal{M}_n} \mathrm{wt}(M)$
- In some cases, we can also use Cayley's theorem to count the number of perfect matchings
- If $a_{i,j} = \pm 1$ for all edge (i,j), then wt(M)$= \pm 1$

- If we can choose $a_{i,j}$ so that all perfect matchings have weight $+1$ (or all -1) then we could easily count the perfect matchings as $\sqrt{\det(A)}$



Pfaffian Orientation

0	$B_{1,2}$	$B_{1,3}$	$B_{1,4}$
$-B_{1,2}$	0	$B_{2,3}$	$B_{2,4}$
$-B_{1,3}$	$-B_{2,3}$	0	$B_{3,4}$
$-B_{1,4}$	$-B_{2,4}$	$-B_{3,4}$	0

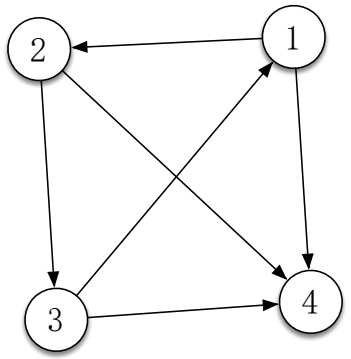


$$Pf(B)=B_{1,2}B_{3,4}-B_{1,3}B_{2,4}+B_{1,4}B_{2,3}$$



0	$B_{1,2}$	$B_{1,3}$	$B_{1,4}$
$-B_{1,2}$	0	$B_{2,3}$	$B_{2,4}$
$-B_{1,3}$	$-B_{2,3}$	0	$B_{3,4}$
$-B_{1,4}$	$-B_{2,4}$	$-B_{3,4}$	0

0	1	-1	1
-1	0	1	1
1	-1	0	1
-1	-1	-1	0



$Pf(B)=B_{1,2}B_{3,4}-B_{1,3}B_{2,4}+B_{1,4}B_{2,3}$

$Pf(A)=3$

Orientation $\vec{E}$ of E:

$$A_{i,j}=\begin{cases}1 & (i,j)\in\vec{E}\\ -1 & (j,i)\in\vec{E}\\ 0 & otherwise\end{cases}$$

- **Pfaffian orientation:** for all perfect matchings M and M′, we have wt(M) = wt(M′)
- If G = (V,E) has a Pfaffian orientation with associated skew-symmetric matrix A, then the number of perfect matchings equals $\sqrt{\det(A)}$



Pfaffian Orientation

- Unfortunately, not all graphs admit a Pfaffian orientation
- Checking whether a general graph has a Pfaffian orientation is also #P-hard

- *[Kastleyn 1963]* Every planar graph has a Pfaffian orientation



Conditions on Pfaffian orientation

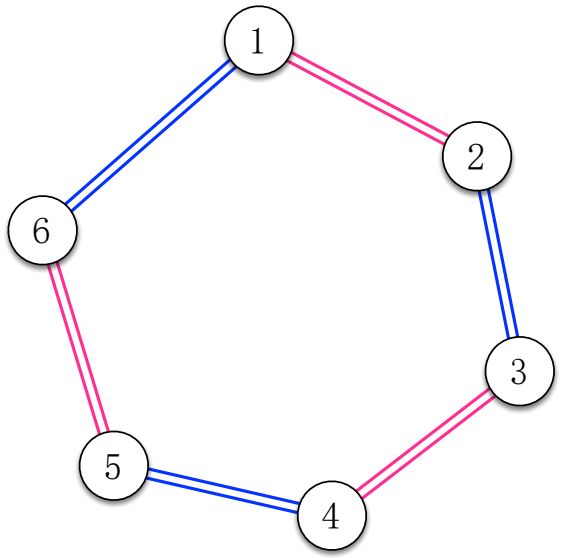
- For Pfaffian orientation, all perfect matchings has the same weight
- Equiv.: Two perfect matchings M,M' different in only one cycle has the same weight

- An even cycle:

wt(M)·wt(M')

$=\mathrm{sgn}((1,2,3,\ldots,2k\text{-}1,2k))\cdot \mathrm{sgn}((2,3,\ldots,2k\text{-}1,2k,1))\cdot A_{1,2}A_{2,3}A_{3,4}\ldots A_{2k-1,2k}A_{2k,1}$

$= -A_{1,2}A_{2,3}A_{3,4}\ldots A_{2k-1,2k}A_{2k,1}$



M, M'



Conditions on Pfaffian orientation

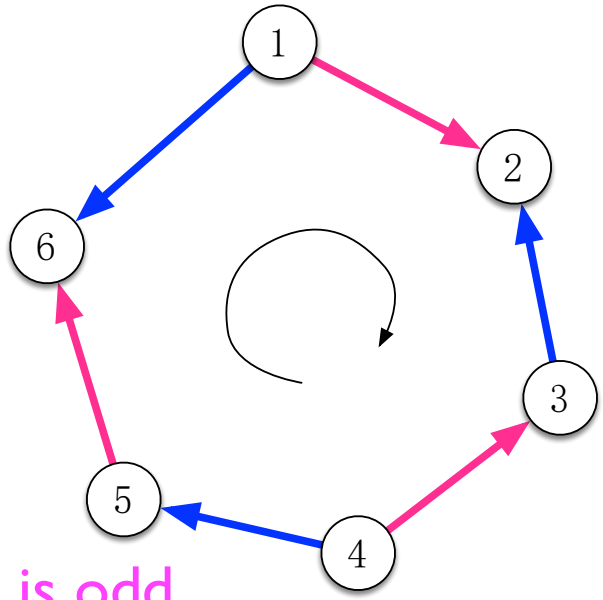
- For Pfaffian orientation, all perfect matchings has the same weight.
- Equiv.: Two perfect matchings M,M' different in only one cycle has the same weight.

- An even cycle:

wt(M)·wt(M')

$=\mathrm{sgn}((1,2,3,\ldots,2k\text{-}1,2k))\cdot \mathrm{sgn}((2,3,\ldots,2k\text{-}1,2k,1))\cdot A_{1,2}A_{2,3}A_{3,4}\ldots A_{2k-1,2k}A_{2k,1}$

$= -A_{1,2}A_{2,3}A_{3,4}\ldots A_{2k-1,2k}A_{2k,1}$

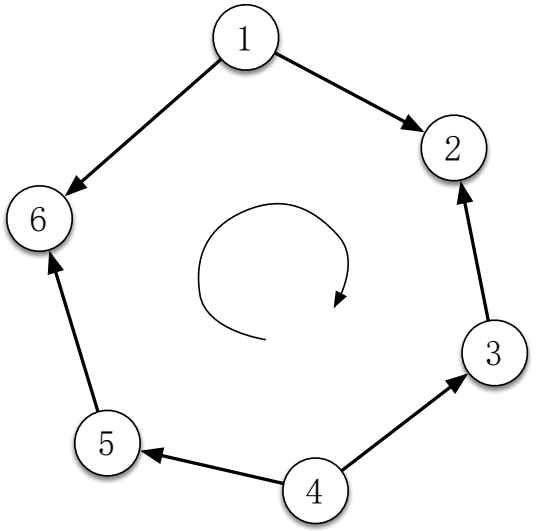


- Given clockwise order, the number of backward oriented edges is odd



Cycles on Pfaffian orientation

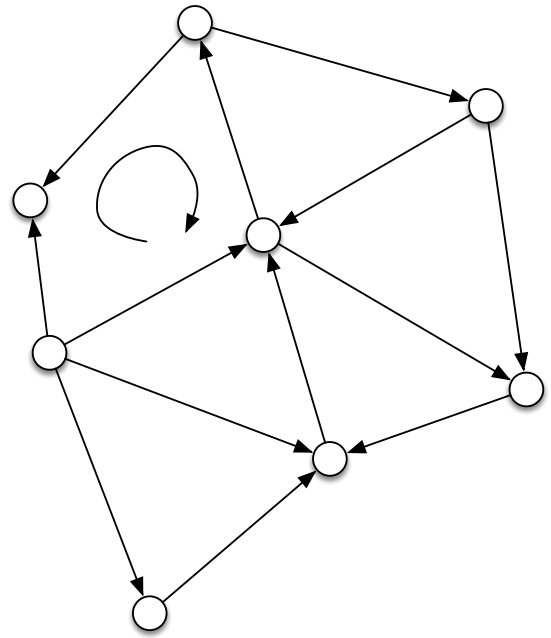
- Pfaffian orientation ⇔ for every even cycle C, if G-C has a perfect matching, then the number of backward oriented arcs along C is odd





Pfaffian orientation on planar graphs

- Let G = (V, E) be a planar graph with a given planar embedding. Suppose G admits an orientation such that for every internal face F, the number of forward edges in clockwise traversal is odd. Then it is a Pfaffian orientation.





Pfaffian orientation on planar graphs

- Let G = (V, E) be a planar graph with a given planar embedding. Suppose G admits an orientation such that for every internal face F, the number of forward edges in clockwise traversal is odd. Then it is a Pfaffian orientation.

- First we prove: for any simple cycle C, the number f of forward clockwise edges plus the number k of vertices inside C is odd

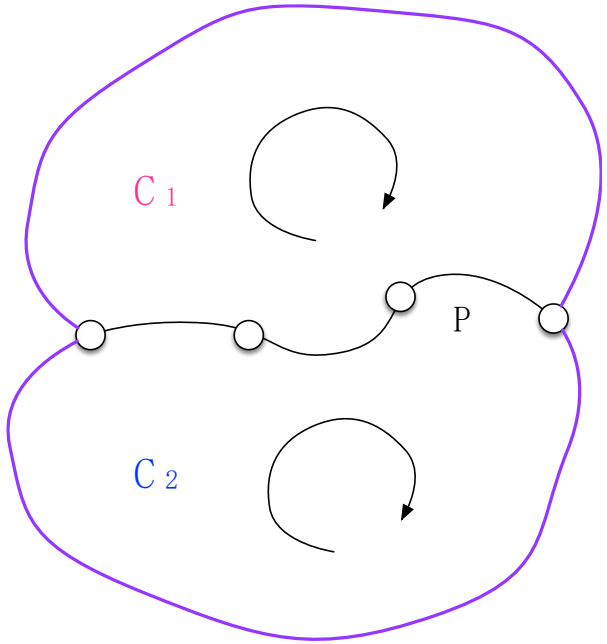
- Single face cycle: condition of the theorem
- For **cycle C** formed by C_1 and C_2 with b internal vertices on the common path P

  For C_1, f_1+k_1 is odd, and for C_2, f_2+k_2 is odd

  For C, $f=f_1+f_2-(b+1)$

  since every edge on P is clockwise either on C_1 or C_2

  $k=k_1+k_2+b$
- Thus, f+k is also odd

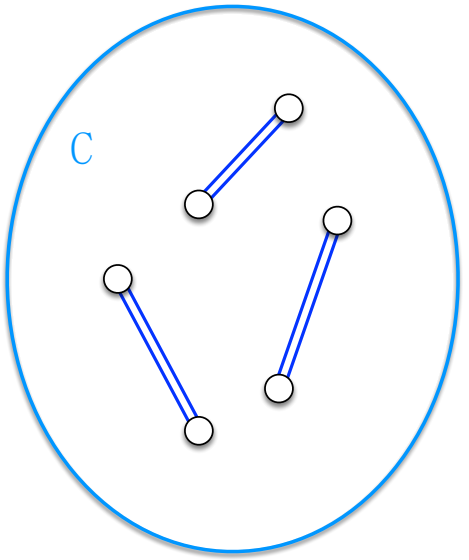


Pfaffian orientation on planar graphs

- Let G = (V, E) be a planar graph with a given planar embedding. Suppose G admits an orientation such that for every internal face F, the number of forward edges in clockwise traversal is odd. Then it is a Pfaffian orientation.

- First we prove: for any simple cycle C, the number f of forward clockwise edges plus the number k of vertices inside C is odd

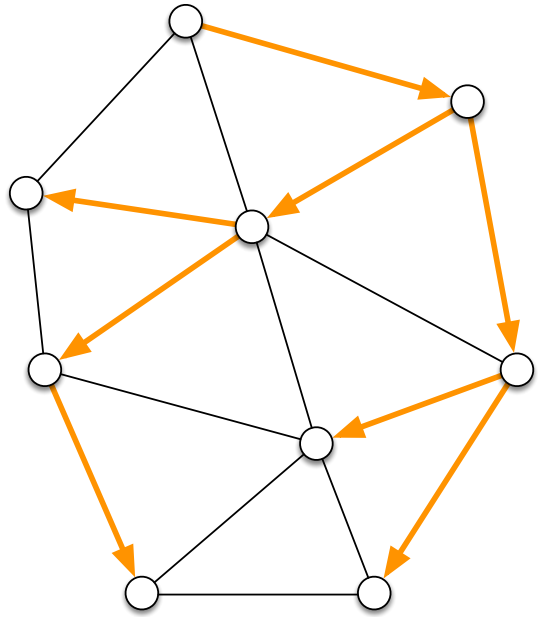
- For any even cycle C satisfying G-C has a perfect matching
  - k is even since it has a perfect matching inside C
  - So f is odd, thus number of backward clockwise edges is also odd
- Thus it is a Pfaffian orientation





Find a Pfaffian orientation on planar graphs

- Pfaffian orientation for planar graph: for every internal face F, the number of forward edges in clockwise traversal is odd
- Find a Pfaffian orientation: like in the proof of Euler's formula
  - first find a spanning tree and gives arbitrary directions
  - then add other edges and pick appropriate directions

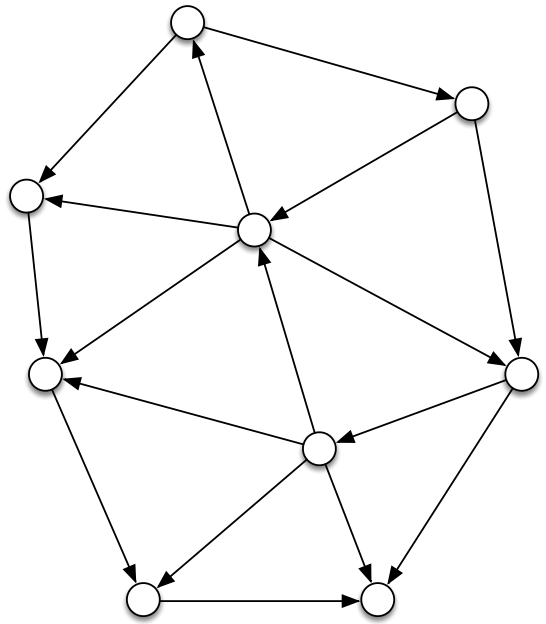




Find a Pfaffian orientation on planar graphs

- Pfaffian orientation for planar graph: for every internal face F, the number of forward edges in clockwise traversal is odd.
- Find a Pfaffian orientation: like in the proof of Euler's formula
  - first find a spanning tree and gives arbitrary directions
  - then add other edges and pick appropriate directions

- Counting perfect matchings in planar graph is in polynomial time
  - Finding a Pfaffian orientation
  - #perfect matchings equals $\sqrt{\det(A)}$
  - "FKT algorithm" by Fisher, Kasteleyn, and Temperley





Other problems?

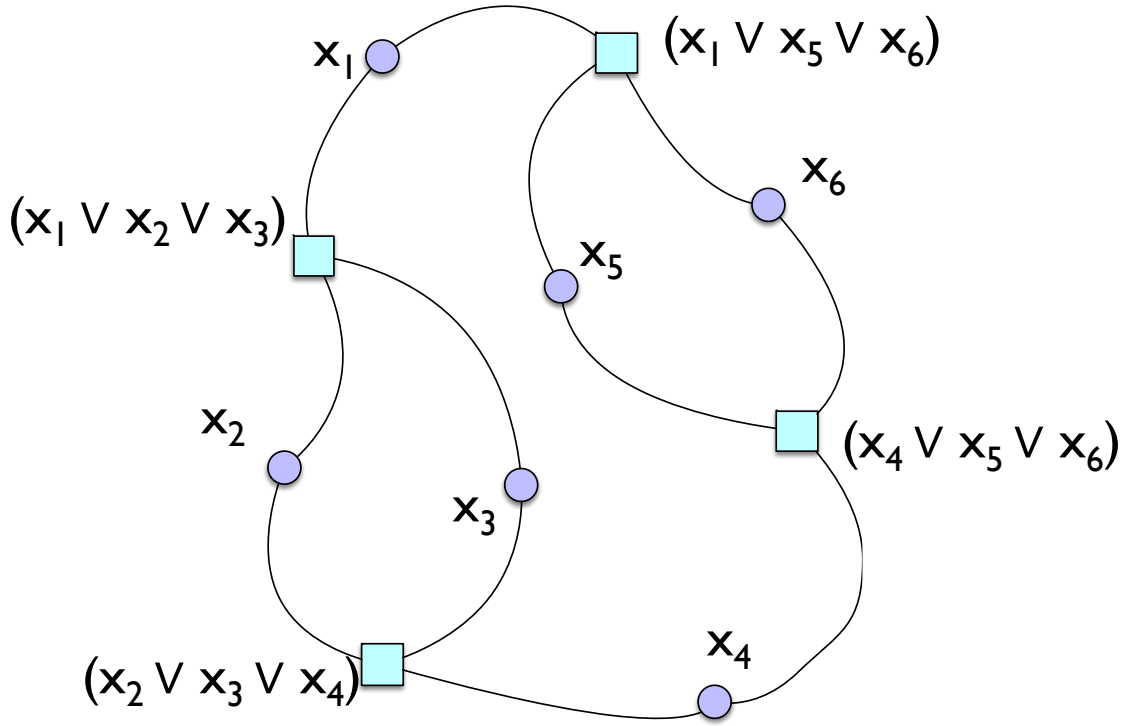
- **#Pl-Rtw-Mon-3CNF:**
  - the number of satisfying assignments of a planar read-twice monotone 3CNF formula

- **Planar:** construct a graph whose vertices are variables x_i and clauses c_j, an edge (x_i,c_j) iff x_i is in c_j, and this graph is planar
- **Read-twice:** every variable appears in two clauses
- **Monotone:** all variables appear positively



#Pl-Rtw-Mon-3CNF: example

- $(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_5 \vee x_6) \wedge (x_2 \vee x_3 \vee x_4) \wedge (x_4 \vee x_5 \vee x_6)$





“Holographic algorithm”

- **#Pl-Rtw-Mon-3CNF:**
  - the number of satisfying assignments of a planar read-twice monotone 3CNF formula
- **Planar:** construct a graph whose vertices are variables x_i and clauses c_j, an edge (x_i,c_j) iff x_i is in c_j, and this graph is planar
- **Read-twice:** every variable appears in two clauses
- **Monotone:** all variables appear positively

- **#Pl-Rtw-Mon-3CNF** is #P-complete *[Xia, Zhang, Zhao 2007]*
- **$\#_7$Pl-Rtw-Mon-3CNF** (module 7) is solvable in polynomial time *[Valiant 2006]*
- **$\#_2$Pl-Rtw-Mon-3CNF** (module 2) is $\oplus$P-complete (also NP-hard)

